

分 类 号: TP399
研究生学号: 2020532107

单位代码: 10183
密 级: 公 开



吉 林 大 学

硕士学位论文

(学术学位)

基于卸载和重计算混合机制的深度学习训练
显存优化算法研究

Research on Memory Optimization Algorithm Based on
Hybrid Mechanisms of Offloading and Recomputing for
Deep Neural Network Training

作者姓名: 周博文

专 业: 计算机系统结构

研究方向: 深度学习神经网络训练系统

指导教师: 魏晓辉 教授

培养单位: 计算机科学与技术学院

2023 年 11 月

基于卸载和重计算混合机制的深度神经网络训练
显存优化算法研究

Research on Memory Optimization Algorithm Based on
Hybrid Mechanisms of Offloading and Recomputing for
Deep Neural Network Training

作者姓名：周博文

专业名称：计算机系统结构

指导教师：魏晓辉 教授

学位类型：工学硕士

答辩日期：2023 年 11 月

摘要

基于卸载和重计算混合机制的深度学习神经网络训练
显存优化算法研究

深度神经网络（Deep Neural Network, DNN）是人工智能领域的关键技术，在多种应用领域展示出卓越的性能。近年来，为了提高 DNN 的准确性，训练数据集的规模和模型的复杂度均大幅增加。由于 DNN 训练必须保留大量的张量，随着模型规模的不断扩大，对 GPU（Graphics Processing Unit, GPU）的显存资源需求显著增加。然而，由于硬件技术限制，GPU 设备的显存扩容速度远低于 DNN 训练作业的发展速度。现有 GPU 显存的硬件限制制约了大规模 DNN 模型的发展。

卸载和重计算是降低训练时显存资源需求的常用机制。卸载是将显存中的张量通过 PCI-E 总线迁移到大容量的 CPU 内存中，重用时再将其预取回 GPU 的显存，但卸载/预取机制可能带来训练过程阻塞的代价。重计算机制只保留部分张量作为检查点，通过从检查点重新计算来满足张量的重用需求，其缺点是引入了张量重复计算的代价。两种机制引入代价的原理不同，分别消耗 PCI-E 总线资源和 GPU 计算资源，存在结合优化的潜力。但不合理的结合策略会引入较高的阻塞和重计算代价，反而对训练性能造成严重的影响。本文主要研究如何在有限显存的条件下高效地结合卸载和重计算，以满足 DNN 训练的显存需求。论文的主要工作如下：

（1）本文提出了混合显存优化统一代价模型，该模型以训练延迟为基准，准确描述卸载/预取和重计算机制的显存优化代价。基于该模型，本文提出了基于卸载和重计算混合机制的显存优化问题（Memory Optimization Problem for GPU Based on Hybrid Mechanisms, HGMOP）。该问题以张量计算的依赖关系和 GPU 显存容量为约束条件，在保证训练作业正确性的前提下，最小化显存优化带来的训练延迟代价。

（2）本文设计了两种算法来求解基于卸载和重计算混合机制的最小化显存优化代价问题。本文设计了基于混合整数规划方法的显存优化算法 MIP-MO（Memory Optimization Algorithm Based on Mixed Integer Programming, MIP-MO）。实验结果表明，与相关工作相比，MIP-MO 算法能够平均提升 DNN 训练性能达

23.5%。为了解决 MIP-MO 算法在处理大规模显存优化问题时求解时间长的问题，本文设计了一种基于深度强化学习的显存优化算法 DRL-MO（Memory Optimization Algorithm Based on Deep Reinforcement Learning, DRL-MO）。DRL-MO 采用了图同构神经网络来提取张量依赖关系，利用动作过滤模块来挑选满足张量依赖约束的操作，利用优势演员-评论家算法实现稳定的强化学习训练。实验结果表明，相比 MIP-MO 算法，DRL-MO 算法开销平均减小 79 倍。与相关工作相比，DRL-MO 算法能够平均提升 DNN 训练性能达 18.1%。

（3）现有训练框架未能提供完善的接口，在实现卸载和重计算的机制方面存在一定的限制，本文设计并实现了轻量的深度学习训练平台 Ant-NN（Ant Neural Network, Ant-NN）。该平台是基于 CUDA 和 CUDNN 实现，支持张量卸载/预取和重计算等灵活的张量管理机制，支持 DNN 作业类型扩展和显存优化策略扩展。本文将所提出的两种显存优化算法集成于 Ant-NN 平台，实现了从全卸载、DRL-MO 到 MIP-MO 的渐进式显存优化策略调整。该平台实现了对用户透明的显存优化策略实施，同时满足用户对训练作业快速启动和高效优化的需求。

关键字：深度神经网络，显存，张量，卸载，重计算

Abstract

Research on Memory Optimization Algorithm Based on Hybrid Mechanisms of Offloading and Recomputing for Deep Neural Network Training

Deep Neural Network (DNN) is a critical technology in the field of artificial intelligence, demonstrating outstanding performance across various application domains. In recent years, both the size of the training dataset and the complexity of the model have increased dramatically in order to improve the accuracy of DNN. Since a large number of tensors must be retained for DNN training, the demand for memory resources of Graphics Processing Unit (GPU) increases significantly as the model size continues to grow. However, due to limitations of hardware technology, the rate of expansion for memory capacity of GPU devices lags far behind the pace of development in DNN training. The hardware limitations for memory capacity of existing GPU hinders the advancement of large-scale DNN.

Offloading and recomputing are common mechanisms for reducing the demand for memory resources of GPU during training. Offloading involves transferring tensors from memory of GPU to large-capacity memory of CPU via the PCI-E bus and prefetching them back to memory of GPU for reuse. However, the offloading/prefetching mechanism may incur the cost of blocking the training process. Recomputing retains only a subset of tensors as checkpoints and satisfies reuse of tensors by recomputing them from these checkpoints. However, it introduces the cost of tensor recomputing. These two mechanisms introduce the cost with different principles, as they consume PCI-E bus resources and computation resources of GPU, respectively, offering potential for optimization through their combination. However, improper combination strategies may introduce high costs in terms of blocking and recomputing, severely affecting training performance. In this paper, we focus on how to efficiently combine offloading and

recomputing under limited memory capacity of GPU to meet the memory requirement of DNN training. The main contributions of this paper are as follows:

(1) In this paper, we propose a unified cost model for GPU memory optimization with hybrid mechanisms, which accurately characterizes the memory optimization costs associated with offloading/prefetching and recomputing mechanisms in terms of training latency. Based on this model, the paper formulates the Memory Optimization Problem for GPU Based on Hybrid Mechanisms of offloading and recomputing(HGMOP).It is constrained by dependencies of tensor computation and memory capacity of GPU. The objective of the HGMOP is to minimize the training latency costs incurred by memory optimization, while ensuring the correctness of the training job.

(2) In this paper, two algorithms are designed to solve the problem of minimizing the cost of memory optimization based on hybrid mechanisms of offloading and recomputing. In this paper, we design the Memory Optimization Algorithm Based on Mixed Integer Programming (MIP-MO). Experiments demonstrate that MIP-MO is able to improve the DNN training performance by up to 23.5% on average compared with related work. In order to solve the problem of long time of solution when dealing with memory optimization problems of large-scale DNN in MIP-MO, we introduce Memory Optimization Algorithm Based on Deep Reinforcement Learning(DRL-MO). DRL-MO leverages a graph isomorphic network to extract dependencies of tensors, utilizes an action filtering module to select operations that satisfy constraints of dependencies of tensors, and employs the Advantage Actor-Critic algorithm to achieve stable training of reinforcement learning. Experiments reveal that, in comparison to the MIP-MO, DRL-MO reduces overhead of solution by a factor of 79 on average. Compared with related work, DRL-MO is able to improve DNN training performance by up to 18.1% on average.

(3) Existing training frameworks lack comprehensive interfaces and have certain limitations in implementing offloading and recomputing mechanisms. In this paper, we design and implement Ant Neural Network(Ant-NN), a lightweight deep learning training platform. This platform is implemented based on CUDA and CUDNN, offering flexible tensor management mechanisms, including offloading/prefetching and

recomputing of tensors. It also supports extensions for various types of DNN job and memory optimization strategies. In this paper, we integrate these two memory optimization algorithms proposed into the Ant-NN platform, enabling progressive adjustment of memory optimization strategies from full offloading, DRL-MO to MIP-MO. The platform provides transparent memory management to users while meeting their demands for fast startup and efficient optimization of training jobs.

Keywords: Deep Neural Network, Memory of GPU, Tensor, Offloading, Recomputing

目 录

第 1 章 绪论.....	1
1.1 研究背景与意义.....	1
1.1.1 深度神经网络训练.....	1
1.1.2 显存墙问题.....	3
1.2 国内外研究现状.....	4
1.3 本文主要内容.....	6
1.4 本文结构安排.....	7
第 2 章 相关工作	8
2.1 卸载机制.....	8
2.1.1 张量传输架构和实现.....	8
2.1.2 张量卸载优化策略.....	9
2.2 重计算机制.....	11
2.3 卸载和重计算相结合的机制	12
2.4 混合整数规划和深度强化学习	13
2.5 本章小结.....	16
第 3 章 基于混合整数规划的显存优化算法	17
3.1 训练数据流图与问题描述	17
3.2 混合显存优化统一代价模型	18
3.2.1 重计算代价建模.....	18
3.2.2 卸载和预取代价建模.....	19
3.3 显存管理约束建模.....	20
3.3.1 张量计算依赖关系的约束.....	20
3.3.2 张量卸载和预取的约束.....	21
3.3.3 张量的显存限制.....	22
3.4 混合机制显存优化问题.....	24
3.5 算法设计.....	25
3.6 本章小结.....	26
第 4 章 基于深度强化学习的显存优化算法	27
4.1 问题描述和算法概述.....	27
4.2 张量状态嵌入.....	28
4.3 张量动作空间设计.....	29
4.4 ACTOR 与 CRITIC 网络设计	32
4.5 优势演员-评论家训练算法	32
4.6 本章小结.....	35

第 5 章 训练平台和实验结果分析	36
5.1 ANT-NN 训练平台	36
5.1.1 Ant-NN 训练平台特性	37
5.1.2 渐进式显存优化策略调整	38
5.2 实验环境与对比用例参数	38
5.3 实验结果与分析	40
5.3.1 MIP-MO 的结果与分析	40
5.3.2 DRL-MO 的结果与分析	43
5.3.3 算子优化	44
5.4 本章小结	45
第 6 章 总结与展望	46
6.1 工作总结	46
6.2 工作展望	47
参考文献	48

第 1 章 绪论

1.1 研究背景与意义

目前，深度神经网络（Deep Neural Network, DNN）作为解决复杂问题的前沿技术，在很多应用中表现出了优越的性能，甚至超过了人类的水平，例如图像识别^[1,2]，推荐系统^[3,4]，语音识别^[5,6]，自然语言处理^[7,8]等等。训练 DNN 涉及大量的张量计算和存储。张量是一种多维数组，支持常见的线性代数的计算，如张量划分，按位读写，矩阵相乘等等。DNN 模型训练对硬件资源有很大需求，尤其是计算和内存资源。由于 GPU（Graphics Processing Unit, GPU）设备强大的计算能力，研究人员经常使用 GPU 来训练 DNN。然而，在模型训练时，张量数据会在前后向传播中重用，这就导致大量的 GPU 显存被用来长时间保留张量数据^[9-11]。为了提升 DNN 解决问题的能力，研究人员逐年拓展深度神经网络的规模，出现了大规模的 DNN，如 GPT3^[7]，T5^[12]等等，其训练显存需求显著增加。然而，GPU 设备扩容速度落后于 DNN 的发展速度。从 2018 到 2022 年，DNN 的权重规模拓展了约 1 万倍，但 GPU 的显存容量仅增加了 3 倍。所以，显存成为了阻碍 DNN 模型规模进一步扩展的关键瓶颈^[9,10,13,11,14,15]。

1.1.1 深度神经网络训练

DNN 是由多个隐藏层组成的复杂网络结构，其中隐藏层的权重是 DNN 解决问题的关键因素。为了获得优秀的网络能力，DNN 通过训练寻找最佳权重，使其输出尽可能地逼近目标值。但在训练过程中，DNN 的输出与目标值之间存在损失 *Loss*。为了减少 *Loss*，训练通常利用梯度下降法^[16]调整隐藏层的权重。该方法根据 *Loss* 利用反向传播算法^[17]计算权重的梯度，并沿着梯度下降的方向调整权重。为获得最优权重，网络设计者利用大规模的训练数据，迭代地更新权重，直至 *Loss* 收敛。在每次训练迭代时，训练以一批训练数据作为输入张量，经过前向传播和后向传播计算权重的梯度。其中，前向传播负责计算 *Loss*，而后向传播负责计算权重的梯度。

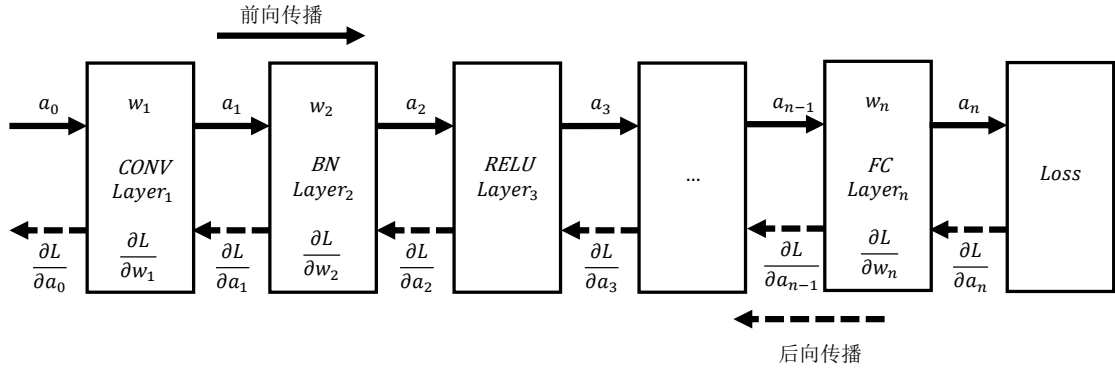


图 1.1 深度卷积神经网络训练示例

$Loss$ 是 DNN 输出与目标值之间的差距。目标值是训练数据中已知的，而 DNN 的输出需要通过前向传播计算得到，如图 1.1 中的实线方向所示。图 1.1 呈现了一个深度卷积神经网络的训练示例，它包括了卷积层（CONV），批标准化层（BN），激活层（RELU），全连接层（FC）等。在前向传播中，DNN 以一批训练数据作为输入张量。首先，在第一个隐藏层，输入张量 a_0 与权重张量 w_1 进行卷积计算。该层提取训练数据的特征，得到其输出，即特征映射张量 a_1 。接着，第一层的输出作为第二层的输入，依次进行计算。直至最后一层，前向传播得到整个 DNN 的输出张量 a_n 。然后，前向传播通过损失函数计算输出张量 a_n 与目标值之间的 $Loss$ 。

根据前向传播的 $Loss$ ，后向传播利用反向传播算法^[17]计算权重梯度，即图 1.1 中虚线所示的方向。具体来说，反向传播算法利用链式法则依次计算出每个隐藏层的权重梯度和特征映射梯度。首先，后向传播通过对损失函数求导得到 DNN 输出 a_n 的导数，即特征映射梯度张量 $\frac{\partial L}{\partial a_n}$ 。然后，基于链式法则，后向传播得到最后一层的输入张量 a_{n-1} 的特征映射梯度 $\frac{\partial L}{\partial a_{n-1}}$ 和权重张量 w_n 的权重梯度 $\frac{\partial L}{\partial w_n}$ ，如公式 1.1 和 1.2 所示。在计算权重梯度时， $\frac{\partial L}{\partial w_n}$ 依赖 $N-1$ 层前向传播输出的特征映射张量 a_{n-1} 。接着，后向传播将张量 $\frac{\partial L}{\partial a_{n-1}}$ 传递回 $N-1$ 层，继续计算特征映射梯度和权重梯度。直到第一层，后向传播完成整个 DNN 的权重梯度计算。然后，训练使用权重梯度更新 DNN 的权重，以下一批训练数据作为新的输入张量，重复前向传播和后向传播。

$$\frac{\partial L}{\partial a_{n-1}} = \frac{\partial L}{\partial a_n} \otimes w_n \quad (1.1)$$

$$\frac{\partial L}{\partial w_n} = \frac{\partial L}{\partial a_n} \otimes a_{n-1} \quad (1.2)$$

1.1.2 显存墙问题

权重梯度的计算依赖前向传播的特征映射,DNN 的训练必须使用大量的 GPU 显存来保存它们,甚至超出了显存的容量。为了突破显存墙问题,有必要对训练中的数据进行显存管理。DNN 训练的数据是以张量为粒度进行存储,表 1.1 展示了三个 DNN 训练时张量占用的显存资源,它们的训练数据集都是 ImageNet^[18]。从表中所知,训练中的显存主要被四类张量占用,它们分别为网络权重,特征映射,权重梯度和特征映射梯度。

表 1.1 DNN 训练显存占用分析

DNN 类型	训练显存	网络权重 (比例)	特征映射 (比例)	权重梯度 (比例)	特征映射梯度 (比例)
VGG19 ^[19] batch size 160	21612MB	606MB (2.8%)	20276MB (93.81%)	606MB (2.8%)	124MB (0.57%)
ResNet152 ^[20] batch size 128	22369MB	594MB (2.65%)	21034MB (94.03%)	594MB (2.65%)	147MB (0.65%)
Inception V4 ^[21] batch size 64	25755MB	550MB (2.13%)	24547MB (95.3%)	550MB (2.13%)	108MB (0.41%)

网络权重张量: DNN 是由多层隐藏层构成的层次结构,隐藏层的权重是其解决问题的关键。在前向传播中,隐藏层使用权重与其输入进行计算,如卷积计算,矩阵计算等等。而在后向传播计算时,训练使用了以链式法则为基础的反向传播算法,必须计算特征映射梯度,而特征映射梯度的计算依赖于网络权重。因此,网络权重在前后向传播中都被访问,它们的生命周期较长。然而,权重张量占用了较少比例的显存,如表 1.1, VGG19^[19], ResNet152^[20], Inception V4^[21]的权重占比分别为 2.8%, 2.65%, 2.13%。

特征映射张量: 特征映射是隐藏层的输出。在前向传播中,它是隐藏层使用权重和其输入计算得到的,作为后续隐藏层的输入。在后向传播时,隐藏层权重梯度的计算会依赖于特征映射。特征映射在前后向传播中都被访问,它们的生命周期较长。而且它们占用了较多比例的显存,如表 1.1, VGG19^[19], ResNet152^[20], Inception V4^[21]的特征映射占比分别为 93.81%, 94.03%, 95.3%。然而,在特征映射张量的生命周期中,其大部分时间是空闲的。图 1.2 是本文使用 Inception V4 模型训练收集的数据,它展示了 Inception V4 训练中部分特征映射张量的访问和空闲时间,特

征映射按产生的顺序排序。从图中可知，多数特征映射的空闲时间占据了 90% 以上的生命周期。

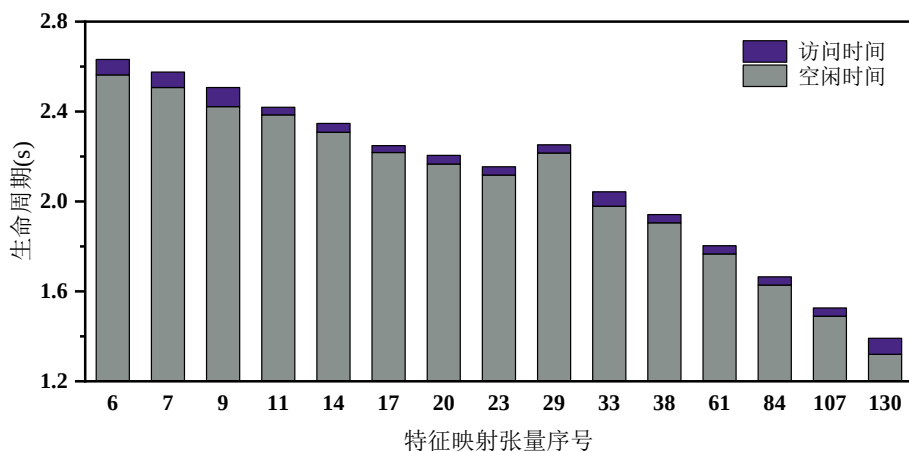


图 1.2 Inception V4 特征映射张量的访问和空闲时间

权重梯度张量：训练利用权重梯度调整隐藏层的权重使得 DNN 的输出逼近目标值。在训练常用的反向传播算法中，权重梯度只能在后向传播计算得到。在所有隐藏层的权重梯度计算完成后，它们会用于权重的更新。权重梯度的生命周期短，而且占用了较少比例的显存，如表 1.1，VGG19^[19]，ResNet152^[20]，Inception V4^[21] 的权重梯度占比分别为 2.8%，2.65%，2.13%。

特征映射梯度张量：反向传播算法是基于链式法则的，必须计算特征映射梯度。与权重梯度相同，特征映射梯度只在后向传播中计算得到。特征映射梯度的生命周期短，且占用了较少比例的显存，如表 1.1，VGG19^[19]，ResNet152^[20]，Inception V4^[21] 的特征映射梯度仅占用了 0.57%，0.65% 和 0.41% 的显存资源。

训练中的这四类张量占用了大量的显存，其中，特征映射张量的生命周期长，且占用的比例最大，有必要对它们进行显存管理。而且，特征映射张量在其生命周期的大部分时间内都是空闲的，它们具有很高的优化潜力。研究人员可以利用这些特性来降低训练的显存需求。

1.2 国内外研究现状

特征映射张量具有很高的优化潜力。近年来，国内外的研究人员主要针对这一问题提出了不同的机制，以突破显存墙瓶颈。典型的相关工作主要围绕张量卸载、张量重计算，以及两者相结合的机制开展工作。

张量卸载：由于特征映射张量数据在前向传播和后向传播之间一直存储于显存中，但在其大部分生命周期中都是空闲的。因此，参考虚拟内存机制的设计思路，将这部分张量通过 PCI-E 总线临时卸载到大容量的系统内存^[22-25,9]，便可以释放大量的显存，用于训练任务。在这部分张量需要被再次使用前，再通过预取机制传输回 GPU 的显存，便可以保证训练的正常运行。VDNN^[22]是首个使用张量卸载机制降低显存需求的相关工作。

张量卸载虽然能够释放大量的显存空间，但系统 PCI-E 总线有限的传输速度可能导致张量未能在显存不足前及时被卸载到系统主存或是在后向传播依赖前及时被预取到显存，从而导致训练的额外阻塞时间。为了减少阻塞代价，LMS^[9]选择了重用距离大于一定范围的特征映射进行卸载和预取。但是当显存竞争激烈时，LMS 仍然会引入大量的训练阻塞时间。

张量重计算：重计算机制保留了一部分特征映射张量作为检查点，通过释放其它张量以降低显存需求^[26,10,13]。在重用被释放的张量时，重计算机制通过从检查点重新计算来再生这些张量。然而，由于引入了额外的计算，重计算会延长训练时间。但不同的张量需要不同的显存资源和计算耗时，保留不同特征映射会引入不同的重计算代价。为降低重计算代价，研究人员采用了不同的策略。陈天奇等人^[26]通过动态规划搜索重计算的策略，而 DTR^[13]通过启发式的标准来选择需要重计算的张量。Checkmate^[10]将重计算问题抽象为整数规划问题，以最小化重计算代价为目标，并通过现有的求解器来搜索重计算策略。然而，当显存需求较高时，这些策略仍然导致大量的重计算时间。

卸载和重计算相结合：在显存竞争激烈时，单一显存优化机制对训练引入了显著的延迟。由于这两种机制分别依赖于 PCI-E 总线资源和 GPU 计算资源，它们存在结合优化的潜力。张量通过卸载和重计算降低显存需求时引入了不同的优化代价，结合机制为不同的张量分配适当的卸载或重计算机制，以降低显存优化引入的代价。因此，SuperNeurons^[27]根据隐藏层的类型对特征映射张量配置卸载和重计算策略。与此不同，Capuchin^[11]发现以隐藏层为粒度的结合策略存在性能浪费，并提出了以张量为粒度的结合策略。然而，Capuchin 利用启发式的方法为张量配置显存优化机制，可能导致不当的结合策略，从而降低训练性能。

其它：研究人员也提出了其它的机制突破显存墙，例如压缩^[28]，分布式训练^[29,30]等。由于张量数据的稀疏性给数据稀疏化方法^[28]提供了可能，采用梯度压缩^[31]

和量化^[32]等方法能够显著降低张量的显存资源需求。另一方面，模型并行^[30]和分布模型状态的数据并行^[33]都可以实现大模型 DNN 的分布式存储和训练，从而横向扩展多个 GPU 设备协同完成计算。这些相关工作与本文研究的内容并不冲突，结合这些优化工作，将有助于进一步缓解深度学习训练作业的显存墙问题^[34,14,35]。例如，CSWAP^[34]利用压缩来加速卸载和预取的传输速度，MPress^[35]将卸载和重计算引入了分布式训练等。

综上所述，降低训练的显存需求的方式主要是卸载，重计算以及两者的结合。然而，采用单一优化机制可能会显著影响训练性能，而多机制的结合为性能的提升带来了可能性。但如何在有限的显存条件下高效地结合这两种机制是当前面临的挑战。

1.3 本文主要内容

现有的结合工作利用启发式的方法获得结合策略，这种方式容易给张量配置不当的显存优化机制，降低训练性能。针对此问题，本文设计了两种显存优化算法来寻找高效的卸载和重计算结合策略，论文的主要工作如下：

(1) 本文提出了混合显存优化统一代价模型，该模型以训练延迟为基准，准确描述卸载/预取和重计算机制的显存优化代价。基于该模型，本文提出了基于卸载和重计算混合机制的显存优化问题 (Memory Optimization Problem for GPU Based on Hybrid Mechanisms, HGMOP)。该问题以张量计算的依赖关系和 GPU 显存容量为约束条件，在保证训练作业正确性的前提下，最小化显存优化带来的训练延迟代价。

(2) 本文设计了两种算法来求解基于卸载和重计算混合机制的最小化显存优化代价问题。本文设计了基于混合整数规划方法的显存优化算法 MIP-MO (Memory Optimization Algorithm Based on Mixed Integer Programming, MIP-MO)。实验结果表明，与相关工作相比，MIP-MO 算法能够平均提升 DNN 训练性能达 23.5%。为了解决 MIP-MO 算法在处理大规模显存优化问题时求解时间长的问题，本文设计了一种基于深度强化学习的显存优化算法 DRL-MO (Memory Optimization Algorithm Based on Deep Reinforcement Learning, DRL-MO)。DRL-MO

采用了图同构神经网络来提取张量依赖关系，利用动作过滤模块来挑选满足张量依赖约束的操作，利用优势演员-评论家算法实现稳定的强化学习训练。实验结果表明，相比 MIP-MO 算法，DRL-MO 算法开销平均减小 79 倍。与相关工作相比，DRL-MO 算法能够平均提升 DNN 训练性能达 18.1%。

(3) 现有训练框架未能提供完善的接口，在实现卸载和重计算的机制方面存在一定的限制，本文设计并实现了轻量的深度学习训练平台 Ant-NN (Ant Neural Network, Ant-NN)。该平台是基于 CUDA 和 CUDNN 实现，支持张量卸载/预取和重计算等灵活的张量管理机制，支持 DNN 作业类型扩展和显存优化策略扩展。本文将所提出的两种显存优化算法集成于 Ant-NN 平台，实现了从全卸载、DRL-MO 到 MIP-MO 的渐进式显存优化策略调整。该平台实现了对用户透明的显存优化策略实施，同时满足用户对训练作业快速启动和高效优化的需求。

1.4 本文结构安排

本文分为六个章节对研究内容进行详细介绍。

第一章为绪论。首先，该章介绍了详细的 DNN 训练过程，并分析了显存墙问题。然后，该章概述了国内外显存墙的研究现状，给出了本文的主要内容和结构。

第二章为相关工作。首先，该章介绍了降低显存需求的主要机制，即卸载，重计算和两者结合，并详细分析了现有工作的不足。然后，该章介绍了本文算法设计使用的两种方法，它们分别为混合整数规划和强化学习。

第三章为基于混合整数规划的显存优化算法。首先，该章对不当结合的问题进行了详细描述。然后，该章介绍了混合显存优化统一代价模型，并介绍了混合机制的显存优化问题。最后，该章分析了问题的复杂度并设计了 MIP-MO 算法。

第四章为基于深度强化学习的显存优化算法。首先，该章介绍了 MIP-MO 算法的主要劣势，并概述了基于深度强化学习的显存优化算法 DRL-MO。接着，该章详细地介绍了 DRL-MO 算法。

第五章为训练平台和实验结果分析。首先，该章介绍了本文设计并实现的 DNN 训练平台 Ant-NN。然后，该章介绍了实验的环境和对比用例参数的设置。最后，通过对比实验分析了 MIP-MO 和 DRL-MO 算法的优势。

第六章总结全文并展望未来。

第2章 相关工作

2.1 卸载机制

2.1.1 张量传输架构和实现

由于 CPU 的内存容量通常较大且成本相对较低，卸载机制可将 CPU 内存作为 GPU 的缓存池，临时存储显存中的张量。在现代计算机系统结构中，GPU 通常作为 CPU 的外设设备，二者通过 PCI-E 总线相连，如图 2.1 所示。作为外设设备，GPU 无法直接访问 CPU 内存中的数据，必须通过 PCI-E 总线将数据传输到显存。当前，由于硬件成本和技术限制，CPU 的内存往往比 GPU 的显存容量更大，例如在云服务器中，CPU 的内存容量可以达到 512GB，甚至几 TB，而当前英伟达的 GPU 产品中，显存容量最大的 A100 仅为 80GB。基于此，CPU 的内存可以作为 GPU 的缓存池。当发生显存不足时，卸载机制^[22,23,9,36]将 GPU 显存中的张量通过 PCI-E 总线迁移到 CPU 的内存，以减少显存占用，如图 2.1 中红线所示。当 GPU 程序需要访问被卸载的张量时，预取机制通过 PCI-E 总线将张量迁移回 GPU 显存，如图 2.1 中绿线所示。

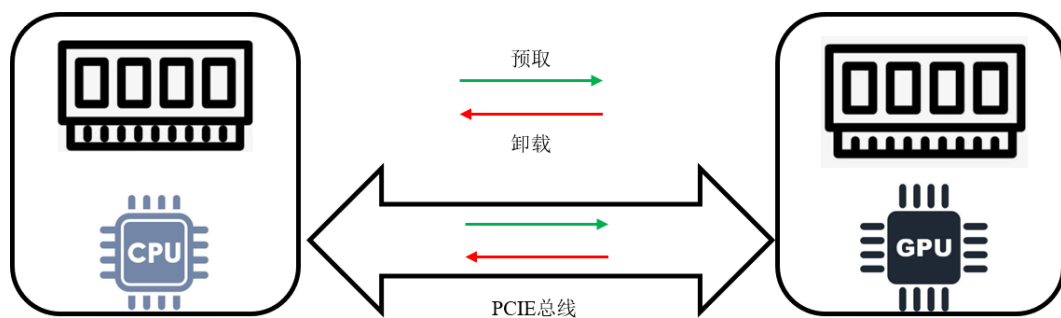


图 2.1 CPU 和 GPU 的系统组成结构

卸载和预取的实现方式包括串行方式和并行方式，两者对训练性能有着不同的影响。这两种实现方式利用了 GPU 驱动提供的相关功能，即 CUDA 流。CUDA 流是 CUDA 程序的抽象概念，用来辅助用户设计并行程序。在 GPU 设备上运行的 CUDA 程序可以视为流中的一系列任务。其中，流内任务必须串行执行，而流与流之间的任务可以并行执行^[37]。本文通过利用 CUDA 流来实现卸载和预取任务。在主流的 GPU 中，由于 DMA（Direct Memory Access，DMA）控制器的辅助，GPU

上的计算、卸载和预取任务可以并行执行。然而，这些任务是否并行执行与 CUDA 流的利用方式有关，例如，图 2.2 所示的卸载/预取的串行和并行实现。在串行的方式中，传输和计算任务都在一个 CUDA 流中排队，等待 GPU 驱动的调度。该方式会将所有的传输时间暴露在训练过程中，导致严重的训练延迟。而采用并行方式的卸载和预取利用多个流来实现^[11]。其中，一个流中排队计算任务，而其他流中排队传输任务。常用的是利用三个 CUDA 流来实现计算，卸载和预取的并行，如图 2.2 所示的 *b* 和 *c* 子图。由于流之间的任务可以并行运行，并行方式的传输时间可以被计算过程部分隐藏，从而减少在训练过程中暴露的传输时间。

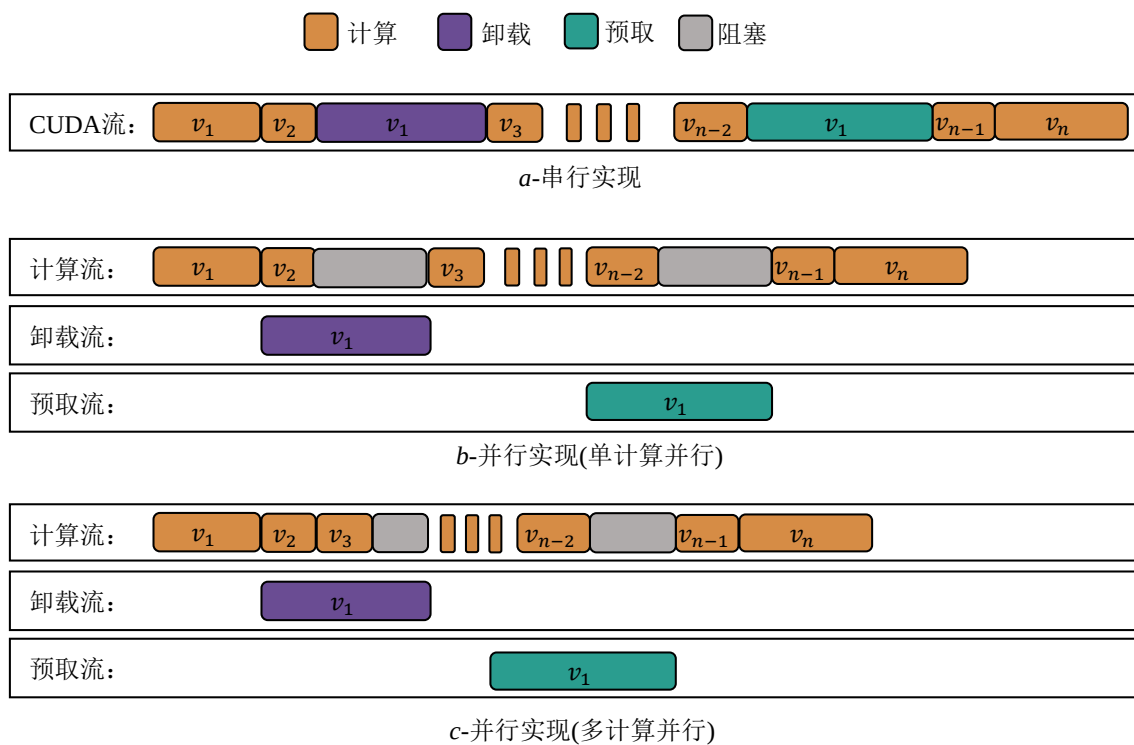


图 2.2 卸载/预取的串行与并行实现

2.1.2 张量卸载优化策略

由于特征映射张量在训练时具有较高的空闲时间比例，在前向传播时，卸载机制^[22,23,9,36]把 GPU 显存中空闲的特征映射张量并行地卸载到 CPU 主存，当在后向传播访问特征映射时并行地预取回 GPU 显存。由于有限的 PCI-E 总线带宽，卸载和预取操作可能会阻塞训练进程。VDNN^[22]为了减少训练的显存占用，在前向传播的卷积层计算阶段卸载其输入张量。然而，为了保证卸载的完成，VDNN 在每个计算任务结束时强制等待卸载任务的完成。例如，在图 2.2 中的 *b* 子图，当 v_2 隐

隐藏层计算结束时, VDNN 需要强制等待 v_1 隐藏层输出特征映射卸载的完成。这阻塞了 v_3 隐藏层的计算, 引入了卸载阻塞时间。当重用被卸载的张量时, VDNN 在前一个隐藏层计算时将其取回。例如, 在图 2.2 中的 *b* 子图, 当依赖 v_1 的输出张量的 v_{n-1} 隐藏层计算之前, VDNN 需要在 v_{n-2} 隐藏层计算时预取 v_1 张量, 这会阻塞了 v_{n-1} 隐藏层的计算, 引入了预取阻塞时间。VDNN++^[23]认为多数隐藏层的计算耗时短于传输特征映射的时间。不同于 VDNN, VDNN++将传输任务与多个计算任务并行, 减少其阻塞的时间, 如图 2.2 中的 *c* 子图。这两种方法都致力于最小化显存占用, 但在一定的显存容量下, 它们存在冗余的卸载和预取操作, 导致了不必要的阻塞, 进而恶化了训练性能。

为了减少引入的阻塞代价, 先前的研究工作^[9,36,24]谨慎地选择合适的张量进行卸载和预取。由于不同的张量具有不同的卸载预取阻塞代价, AutoTM^[36]将卸载哪些张量的问题抽象成了整数规划问题。但由于并行卸载和预取的复杂性, AutoTM 对问题进行了简化, 它在计算结束时强制等待卸载和预取的完成, 引入了大量的阻塞时间。

此外, 张量的阻塞代价与其卸载和预取的起始时间相关。如果张量的卸载时间较晚, 可能导致在显存不足之前无法完成卸载, 从而迫使训练等待其完成。如果张量的预取时间较晚, 可能导致在后向传播访问之前未能完成预取, 同样需要等待其完成。然而, 由于预取会增加显存占用, 过早的预取可能导致显存不足。因此, LMS^[9]提出了三个标准来确定选择哪些张量进行卸载, 以及它们的卸载和预取开始时间。首先, 由于张量在产生之后被多次访问, LMS 认为具有适当的产生和访问间隔距离的张量才有资格进行卸载。然而, 过小的间隔距离可能导致卸载大量特征映射, 这会引入大量阻塞时间, 而过大的间隔距离可能导致只有少量张量被卸载, 这会使得释放的显存无法满足训练的显存需求。其次, 对于卸载的开始时间, LMS 选择在张量产生之后立即进行卸载。对于预取的开始时间, LMS 通过预取提前距离来确定。然而, 过小的提前距离会引入大量的阻塞时间, 而过大的提前距离会导致显存不足。除此之外, 由于张量会被多次访问, 若在每次访问之前均进行预取, 这将导致大量阻塞时间。LMS 认为具有短间隔距离的访问可以共享一次预取。对于这三个标准, LMS 提出了一种基于二分查找的自动调整算法搜索产生访问间距, 预取提前距离和预取共享距离。当距离选择过大, 造成显存不足时, LMS 就向更小的距离范围调整。若距离选择过小, 造成性能恶化, LMS 就向更大的距离范围

调整。然而,在需要大量显存的 DNN 训练中,这些工作仍会引入显著的阻塞代价。

2.2 重计算机制

随着 GPU 计算能力的不断增强,张量卸载和预取的阻塞可能比计算更耗时,研究人员又提出了另一种减少显存需求的机制——重计算^[26,10,13]。重计算通过保留部分张量作为检查点,释放其它张量以腾出显存空间。当被释放的张量再次被访问时,从现有的检查点开始重新执行计算再生它们。由于需要通过额外的计算再生被释放的张量,重计算实际上是一种以时间换取空间的机制。在深度神经网络中,每个张量具有不同的资源需求和计算耗时,但大多数张量的计算耗时相对较短。图 2.3 是本文使用 Inception V4 模型训练收集的数据,它展示了 Inception V4^[21]特征映射张量的计算耗时,其中 98.2%的特征映射计算时间是低于 10 毫秒的。对于相对有限的 PCI-E 总线带宽,特征映射张量的卸载和预取传输时间明显更长。图 2.4 是本文使用 Inception V4 模型训练收集的数据,它展示了 Inception V4 中部分特征映射的计算,卸载和预取的耗时,特征映射按产生的顺序排序。从图中可知,多数特征映射张量的卸载预取耗时明显多于计算耗时,且在最坏的情况下,卸载和预取耗时是计算的 16 倍。所以,重计算也是一种相对高效的显存优化技术。

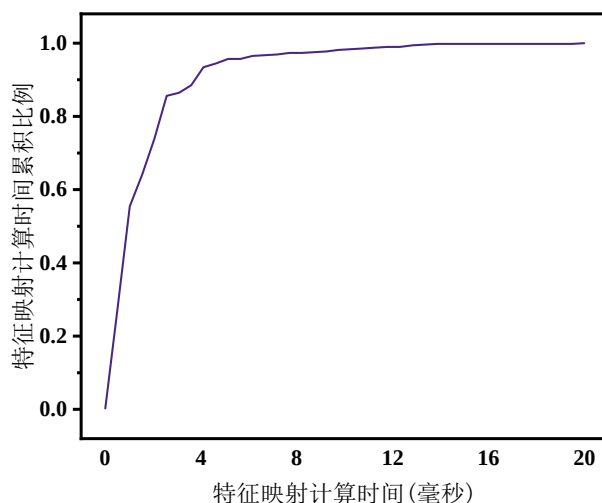


图 2.3 Inception V4 特征映射计算时间概率累积图

保留不同的检查点会引入不同的重计算时间。对于 N 层的神经网络,在训练时,保留特征映射张量的显存需求是 $O(N)$ 。为了将显存需求降低到 $O(\sqrt{N})$,同时减少引入的重计算时间,陈天奇等人^[26]把网络分成了多个部分。每部分包括多个隐藏层组成,但只保留其输入特征映射。对于其它特征映射,每部分都需要重新计

算。但是，陈天奇等人的工作盲目地降低显存，导致了冗余的重计算。

为了减少冗余的重计算，一些工作探究了在有限的显存容量下选择哪些张量作为检查点，以降低重计算代价^[10,13]。选择的标准与张量的特性相关，包括计算耗时和显存占用大小。直观地来说，应该选择显存占用较小但计算耗时较大的张量作为检查点。然而，张量的重计算时间会发生变化。若它依赖的张量被释放，这会引入长链的重计算。而且，张量通过重计算再生之后，可以作为新的检查点，这会减少其它张量的重计算时间。DTR^[13]与先前工作不同，它并不直接选择作为检查点的张量，而是选择被重计算的张量。DTR 定义了量化的选择标准，该标准与张量的“计算时间”，显存占用大小以及多久未被访问相关。在 DNN 训练的过程中，当发生显存不足时，DTR 选择计算时间较小，显存占用较大且未被访问时间较长的张量作为重计算的张量。为了避免出现长链的重计算，DTR 在计算张量的“计算时间”时会考虑与其相关张量的计算时间。与 DTR 不同，Checkmate^[10]把重计算问题抽象为整数规划问题，以最小化重计算代价为目标，并利用现有的求解器解决该问题。尽管这些方法在一定程度上减少了重计算代价，但在需要大量显存时仍然会引入显著的重计算代价。

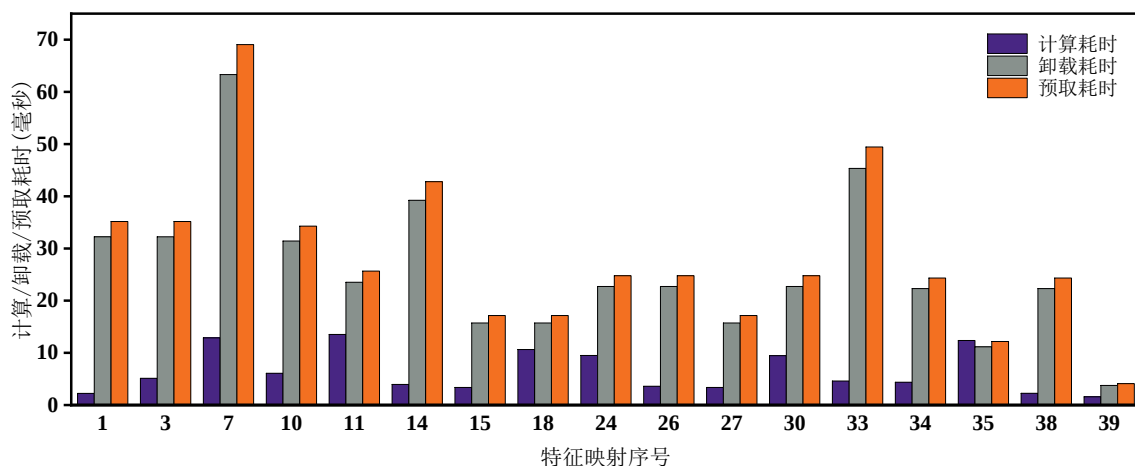


图 2.4 Inception V4 中特征映射的计算，卸载和预取耗时

2.3 卸载和重计算相结合的机制

当显存资源竞争激烈时，单一的显存优化机制会对训练性能带来显著的影响，由于卸载和重计算机制依赖不同的硬件资源，它们存在结合优化的潜力。结合机制^[27,11]为不同的张量分配合适的显存优化机制，以减少显存优化带来的开销，但不当

的结合反而会显著地恶化训练性能。SuperNeurons^[27]认为 DNN 训练是逐个隐藏层进行计算的,只需保证当前在计算的隐藏层的显存需求就可完成训练。对于其它的特征映射,SuperNeurons 通过卸载和重计算释放它们。SuperNeurons 认为不同类型隐藏层的特征映射张量适用于不同的显存优化机制。SuperNeurons 观察到卷积层运行时间长,并不适合重计算。因此,SuperNeurons 卸载和预取卷积层的输出张量。但对于其它的隐藏层如激活层,批标准化层等等,它们的计算耗时相对较短,适用于重计算,SuperNeurons 重计算它们的特征映射张量。虽然 SuperNeurons 将训练的显存降到最低,但在较大的显存容量限制下存在着冗余的卸载和重计算。

为了提高 GPU 训练的性能,减少引入的卸载和重计算代价,金海等人提出了 Capuchin^[11]显存管理系统。Capuchin 为部分张量分配合适的卸载或重计算机制,但张量配置不同的显存优化机制会引入不同的卸载和重计算代价。针对如何给张量分配合适的优化机制, Capuchin 认为有必要为张量的卸载,预取和重计算代价提出量化标准。首先, Capuchin 定义了评估卸载和预取代价的量化标准。由于在每个训练迭代中,张量的访问时间戳是相对稳定的。若张量连续两次的访问间隔越大, Capuchin 认为其越适用于卸载和预取。预取会增加显存的占用,可能造成显存不足。因此, Capuchin 规定了张量提前预取的范围。在该范围之内的预取粗略地估计为无代价,而范围之外预取的代价粗略地估计为张量的传输时间。然后, Capuchin 定义了评估重计算代价的量化标准。Capuchin 认为显存需求越大且重计算耗时短的张量,越适用于重计算。但因为张量的依赖张量会被卸载或重计算,其重计算的代价会被其它张量所影响。Capuchin 根据被卸载和重计算的张量粗略地估计其它张量的重计算代价。最后, Capuchin 设计了一个贪婪的算法,迭代地搜索卸载和重计算的结合策略。它的算法思想是优先对无预取代价的张量分配卸载和预取机制,若释放的显存未满足训练显存需求,则引入重计算。若预取代价高于重计算代价,则张量被重计算,若重计算代价高于预取代价,则张量被卸载和预取。但 Capuchin 未充分考虑两种机制之间的相互影响,存在误判代价的风险,进而为张量配置不当的显存优化机制,产生了低效的结合策略。

2.4 混合整数规划和深度强化学习

整数规划常用于现实的复杂问题,如装箱问题^[38]等。整数规划有三部分组成,

分别为变量,目标和约束。变量在整数规划问题模型中用来建立目标和约束,当变量只存在整数时,该问题模型是经典的整数规划问题模型,而当变量存在整数和非整数时,该问题就是混合整数规划问题模型。目标是现实问题的需要优化的部分,例如最小化成本,最大化收益等等。但根据现实问题的特点,变量的取值要满足一定的条件。所以,变量必须通过约束限制其取值范围。求解整数规划问题就是从有限的方案中找出满足约束条件的最优化方案。目前,寻找最优方案的主流方法是割平面法和分支定界法^[39]。割平面法是通过规划理论淘汰一部分方案,减少求解整数规划问题的计算量。而分支定界算法是通过子问题的上界和下界来进行剪枝,减少计算量。由于很多应用领域需要整数规划,已经有一些工作实现了这两种求解方法,例如 Cplex^[40]和 Gurobi^[41]等。

当前,由于强化学习解决复杂问题的强大能力,它在一些领域取得了卓越的成果^[42-45],例如分布式作业调度^[43]。强化学习是一种自监督的机器学习方法,它根据环境的反馈不断地试错来学习到优秀的策略,如图 2.5 所示。智能体是拥有决策能力的个体,它根据观察到的环境状态,基于自身的策略选择一个动作施加于环境上。环境根据智能体施加的动作,反馈给智能体一个奖励,并转变为下一个状态。强化学习的目的就是通过反复地试错调整自身的策略,使得累积的奖励最大化,从而使智能体拥有强大的决策能力。

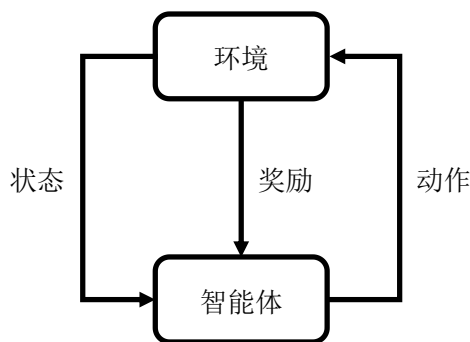


图 2.5 强化学习的基本过程

从环境的初始状态,智能体与环境反复地进行交互直到终止状态,交互过程中产生的状态,动作,奖励等信息是更新策略的重要训练样本。在某个时刻 t ,环境处于状态 s_t ,状态包含了智能体进行决策的主要信息。智能体根据观察到的状态 s_t ,基于自身的策略来选择合适的动作 a_t 。环境基于智能体给出的动作 a_t 反馈给其一个奖励 r_t ,然后根据状态迁移概率 $p(s_{t+1}|s_t, a_t)$ 转移到下一个状态 s_{t+1} 。智能体与环境不断地重复上述过程,直到达到终止状态 s_T 。智能体将交互中的信息记录下来,

形成一个轨迹 τ ，如公式 2.1。智能体不断地与环境进行交互，会产生许多轨迹。强化学习通过这些轨迹，以最大化长期累积奖励 G 为目标，来更新智能体的策略。强化学习获得最佳的策略的方法可以分为基于值函数，策略搜索和演员-评论家的方法。

$$\tau = \{s_1, a_1, p_1, r_1, s_2, a_2, p_2, r_2, \dots, s_T\} \quad (2.1)$$

$$G = \sum_{t=1}^T r_t \quad (2.2)$$

基于值函数的方法获得最佳的策略需要知道当前的策略的好坏，一种评价策略的方法是评估每个动作的价值，如 Q-learning 方法。该方法定义一个状态-动作值函数 $Q(a_i, s_t)$ 来评估策略， $Q(a_i, s_t)$ 表示在 s_t 状态下，若采取 a_i 动作，直到终止状态能获得的累积奖励期望值。若在某个状态下，某个动作累积奖励的期望值越高，表明其价值越高，反之亦反。所以，基于值函数的方法的策略可以定义为公式 2.3，即在每个状态下选择价值最高的动作。

$$\pi(a_i|s_t) = \begin{cases} 1, & a_i = \operatorname{argmax} Q(a, s_t) \\ 0, & \text{其它} \end{cases} \quad (2.3)$$

但基于值函数的方法需要评估每个动作，它适用于具有离散动作的问题，而不适用于具有连续动作的问题，如无人机的转向角度，机器人的手臂控制等等。针对值函数方法的缺陷，研究人员提出了基于策略搜索的强化学习方法。不同于值函数方法，策略搜索方法直接将策略参数化。寻找最佳策略的问题就转化为了策略参数的最优化问题，常见的优化参数的方法是梯度下降法。所以，基于策略搜索的学习方法又被称为策略梯度法。

对于获得最佳策略的方法，值函数和策略梯度法都各有优劣。值函数方法不适用于连续空间的问题，而策略梯度法导致梯度的方差较大，容易收敛到局部极小值。研究人员将值函数与策略梯度方法进行了结合，提出强化学习的演员-评论家方法，例如，深度确定性梯度下降算法 DDPG^[46] (Deep Determined Policy Gradient, DDPG)，异步演员-评论家算法 A3C^[47] (Asynchronous Actor Critic, A3C) 等。

传统强化学习的值函数和策略梯度方法的表征能力有限，不擅长处理复杂的决策问题，但深度学习的出现赋予了值函数和策略强大的表征能力，为强化学习注入了新的动力。不同于传统的强化学习，深度强化学习^[48]采用 DNN 来定义值函数和策略，例如，常见的值函数方法 DQN^[48] (Deep Q-Network, DQN)，优势演员-

评论家算法 A2C^[47] (Advantage Actor Critic, A2C) 等。其中, 基于深度学习的优势演员-评论家算法结合了策略梯度和值函数方法的优势, 它采用两个神经网络, 其一是 Actor 网络, 用来预测某状态下动作的概率分布, 其二是 Critic 网络, 用于评估当前策略选择的动作的优劣。

2.5 本章小结

本章介绍了本文所涉及的相关工作。首先, 本章对主流的显存优化机制卸载, 重计算以及二者的结合进行了详细的介绍, 并详细地介绍了现有的工作, 且分析了它们的不足。接着, 本章介绍了本文的算法设计相关的技术: 混合整数规划和强化学习。

第3章 基于混合整数规划的显存优化算法

3.1 训练数据流图与问题描述

由于 DNN 框架如 Tensorflow^[49], Pytorch^[50]等等的方便与强大, 深度神经网络设计者经常利用它们设计和训练 DNN。与传统的编译器优化相似, 这些主流的框架将 DNN 训练进行抽象, 生成框架的中间表达, 来进行后续的优化。它们将一个训练迭代表示成基于 DAG 形式的训练数据流图。

数据流图是由节点和边组成的, 节点表示的是计算, 如卷积, 矩阵相乘等等, 边代表的是数据依赖。例如, 在图 3.1 中, v_1 计算节点的输出张量被 v_2 , v_{n-1} 和 v_n 节点所依赖。从 v_1 到 v_k 表示训练迭代的前向传播过程, 而从 v_{k+1} 到 v_n 表示训练迭代的后向传播过程。训练框架根据数据流图的拓扑排序, 从第一个节点依次执行计算, 直到最后一个节点, 完成 DNN 所有权重梯度的计算。

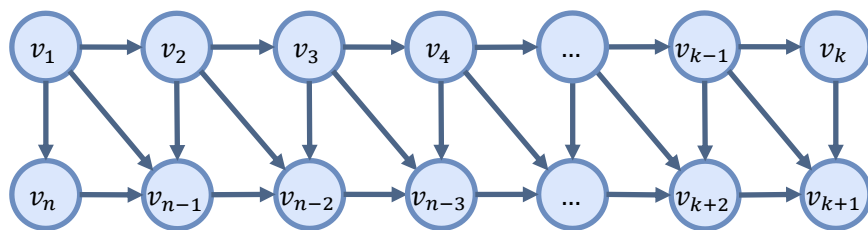


图 3.1 DNN 训练数据流图示例

当 GPU 显存资源无法满足执行数据流图的显存需求时, 结合机制^[27,11]为不同的张量分配合适的显存优化机制来满足执行数据流图的显存需求。由于卸载^[22,23,9,36]和重计算^[13,26,10]依赖不同的硬件资源, 张量通过它们降低显存需求时会引入不同的优化代价。高效的结合配置是在有限的显存内, 为不同的张量配置更低代价的优化机制, 以最小的代价完成训练数据流图的执行。为了给张量分配合理的显存优化机制, 有必要评估张量的卸载, 预取和重计算的代价。因此, Capuchin^[11]定义了启发式的卸载预取和重计算代价量化标准, 利用它们评估张量的显存优化代价, 为张量选择低代价的机制。

然而, Capuchin^[11]未充分考虑两种机制之间的相互影响。首先, 重计算降低了显存占用, 推迟了显存不足的发生, 从而减少了张量的卸载代价。其次, 重计算在张量的后向传播访问之前引入了额外的计算, 导致张量的后向传播访问被推迟,

从而减少了张量的预取代价。此外,被卸载的张量通过预取可以作为其它张量重计算的起点,减少了其它张量的重计算时间。这些因素使得 Capuchin 难以准确评估张量的显存优化代价,有可能为张量分配高代价的显存优化机制。针对此不足,本章建立了精确的混合显存优化统一代价模型,综合考虑了两种机制的优化代价。在此基础上,本章提出了混合机制的显存优化问题 HGMOP。该问题通过添加整数规划约束保证了张量计算,卸载和预取的正确性,并以可用的显存容量作为约束,避免训练发生显存不足。为了解决该问题,本章利用现有的整数规划求解器设计了显存优化算法 MIP-MO。HGMOP 主要由三部分组成:重计算代价建模,卸载/预取代价建模和显存管理约束建模。它们涉及到的变量和符号如表 3.1 所示。

表 3.1 问题模型的变量和符号

变量和符号	描述
$G = (V, E)$	G 是训练数据流图, V 是计算节点的集合, E 是边的集合
$R_{i,j}$	是否在 i 阶段重计算 j 节点
$S_{i,j}$	在 i 阶段时, j 计算节点的输出张量是否可以访问
$O_{i,t,k}$	是否从 t 阶段开始卸载 i 计算节点的输出张量,直到 k 阶段等待卸载完成
$P_{i,t,k}$	是否从 t 阶段开始预取 i 计算节点的输出张量,直到 k 阶段等待预取完成
$U_{t,k}$	t 阶段执行 k 计算节点时 GPU 显存里张量占用的存储大小
$Free_{t,k,i}$	t 阶段执行完 k 计算节点后是否释放 i 计算节点输出张量的显存资源
ofo_i	卸载 i 计算节点的输出张量导致的阻塞时间
pfo_i	预取 i 计算节点的输出张量导致的阻塞时间
C_i	i 计算节点的运行耗时
M_i	i 计算节点输出张量的显存占用大小
B_o	卸载张量的 PCI-E 传输速度
B_p	预取张量的 PCI-E 传输速度
M_{gpu}	可用的显存容量

3.2 混合显存优化统一代价模型

3.2.1 重计算代价建模

DNN 训练可以用训练数据流图 $G = (V, E)$ 来表述。其中, $V = \{v_1, v_2, v_3, \dots, v_n\}$,它是数据流图中节点的集合,表示了该数据流图有 n 个计算节点。而 E 是数据流图

中边的集合，它表示了计算节点之间的依赖关系。每个计算节点 v_i 依赖于其它计算节点 $v_{j<i}$ 的输出张量。HGMOP 把一个训练数据流图划分成多个阶段。例如，图 3.1 中 n 个节点的数据流图可以划分为 n 个阶段。由于引入了重计算^[26,10,13]，在 v_i 阶段计算时，它所依赖的计算节点 v_j 的输出张量或许需要被重计算。因此， $R_{i,j}$ 定义为是否在 v_i 阶段重计算 v_j 节点。而重计算的代价可以定义为公式 3.1， C_j 定义为 v_j 节点的计算耗时。

$$\sum_{i=1}^n \sum_{j=1 \wedge i \neq j}^n R_{i,j} C_j \quad (3.1)$$

3.2.2 卸载和预取代价建模

卸载机制^[22,23,9,36]利用 PCI-E 总线在 CPU 与 GPU 之间迁移张量，但由于有限的 PCI-E 带宽，卸载和预取会阻塞训练的进程，引入了阻塞时间。它的代价如公式 3.2 所示。 $of o_j$ 是卸载 j 计算节点的输出张量需要等待的卸载阻塞时间，而 $pf o_j$ 是预取 j 计算节点的输出张量需要等待的预取阻塞时间，它们的建模细节会在之后介绍。

$$\sum_{j=1}^n of o_j + \sum_{j=1}^n pf o_j \quad (3.2)$$

为了描述卸载和预取的行为，HGMOP 分别定义了 O 和 P 为卸载和预取的变量。HGMOP 规定在每个节点计算的开端，启动卸载或预取张量。 $O_{j,t,k}$ 定义为是否在 t 阶段占用 PCI-E 总线开始卸载 j 计算节点的输出张量，直到 k 阶段等待卸载的完成。类似于卸载， $P_{j,t,k}$ 定义为是否在 t 阶段占用 PCI-E 总线开始预取 j 计算节点的输出张量，直到 k 阶段等待预取的完成。

HGMOP 规定每个阶段只有一个计算节点的输出张量利用 PCI-E 总线进行卸载和预取。若多个计算节点的输出张量进行卸载或预取，它们会竞争 PCI-E 总线资源，引入卸载和预取的阻塞时间无法简单地被计算。因此，阻塞时间即为传输数据的时间与重叠的计算时间之差。公式 3.3 和 3.4 分别为卸载和预取阻塞时间的计算。其中， M_i 定义为 i 节点的输出张量需要的显存大小， B_o 和 B_p 分别定义为卸载和预取 PCI-E 总线的实际传输速度。

$$of o_i = \sum_{t=1}^n \sum_{k=t}^n \max \left(0, \frac{O_{i,t,k} M_i}{B_o} - \sum_{l=t}^k \sum_{r=1}^n R_{l,r} C_r \right), \quad 1 \leq i \leq n \quad (3.3)$$

$$pfo_i = \sum_{t=1}^n \sum_{k=t}^n \max \left(0, \frac{P_{i,t,k} M_i}{B_p} - \sum_{l=t}^k \sum_{r=1}^n R_{l,r} C_r \right), \quad 1 \leq i \leq n \quad (3.4)$$

卸载和预取阻塞时间公式中的取最大值操作是一个非线性运算。它不利于混合整数规划问题的优化，有必要将其转化为线性计算。将取最大值转化线性运算的常用方法是通过引入额外的整数变量和常量来进行线性化。以 i 计算节点输出张量卸载阻塞时间的取最大值运算为例，它的线性化如下公式所示。 u 和 v 是辅助的 0 或 1 整数变量， M 是所引入的正数常量。 z 是一个非整数变量，定义为卸载引入的阻塞时间。公式 3.6 和 3.7 保证了 z 变量大于等于 0 和卸载阻塞时间。当 u 或 v 为 1 时， z 就会分别等于 0 和阻塞时间。

$$u + v = 1 \quad (3.5)$$

$$z \geq 0 \quad (3.6)$$

$$z \geq \frac{O_{i,t,k} M_i}{B_o} - \sum_{l=t}^k \sum_{r=1}^n R_{l,r} C_r \quad (3.7)$$

$$z \leq M(1 - u) \quad (3.8)$$

$$z \leq \frac{O_{i,t,k} M_i}{B_o} - \sum_{l=t}^k \sum_{r=1}^n R_{l,r} C_r + M(1 - v) \quad (3.9)$$

3.3 显存管理约束建模

3.3.1 张量计算依赖关系的约束

一个完整的训练迭代过程是从 v_1 节点执行到 v_n 节点。为了保证计算权重梯度的完整性，公式 3.10 表示 v_1 到 v_n 计算节点都要计算一次。由于每个计算节点 v_i 依赖于已计算过的节点 $v_{j < i}$ 的输出张量，在其依赖张量未计算之前， v_i 节点无法进行计算。公式 3.11 保证了非法的计算无法执行。

$$R_{i,j} = 1, \quad i = j \wedge 1 \leq i, j \leq n \quad (3.10)$$

$$R_{i,j} = 0, \quad 1 \leq i < j \leq n \quad (3.11)$$

每个计算节点都依赖于其它计算节点的输出张量，HGMOP 必须保证每个计算节点执行计算的正确性。而在使用了卸载和重计算释放显存资源之后，在执行某个节点计算时，它所依赖的张量可能并不能被访问，因为依赖张量没通过重计算或

预取进行再生。HGMOP 有必要定义 S 变量表达张量的可访问性。 $S_{i,j}$ 变量定义了 i 阶段时, j 节点的输出张量是否可以访问。为了保证计算执行的正确性, 即在计算之前需要保证依赖张量的访问正确性, 公式 3.12 约束了在执行每个节点的计算时它依赖的张量必须是可访问的或通过重计算再生的。 $DEPS[j]$ 表示 j 计算节点的依赖节点。

$$\begin{aligned} DEPS[j] &= \{k: \langle v_k, v_j \rangle \in E\} \\ R_{i,j} &\leq S_{i,k} + R_{i,k}, \quad k \in DEPS[j] \end{aligned} \quad (3.12)$$

然而, 在训练时, 张量会多次被释放和再生, 它的可访问性也会随之改变。公式 3.13 表示如果在 i 阶段利用重计算或预取再生了 j 计算节点的输出张量, 则 j 计算节点的输出张量在之后的计算中能被其它计算节点访问。在迭代的开始时, 由于没有执行任何节点的计算, 所有节点的输出张量都是不可访问的, 即公式 3.14。

$$S_{i+1,j} \leq S_{i,j} + R_{i,j} + \sum_{t=1}^i P_{j,t,i}, \quad 1 \leq i \leq n-1 \wedge 1 \leq j \leq n \quad (3.13)$$

$$\sum_{j=1}^n S_{1,j} = 0 \quad (3.14)$$

3.3.2 张量卸载和预取的约束

为了简化阻塞时间的计算, HGMOP 规定了每个阶段只有一个计算节点的输出张量利用 PCI-E 总线。公式 3.15 约束了在 m 阶段只有一个计算节点的输出张量在卸载, 公式 3.16 约束了在 m 阶段只有一个计算节点的输出张量在预取。除此之外, 卸载和预取也必须满足以下约束, 公式 3.17 和公式 3.18 约束了不能卸载和预取未计算的张量, 且卸载和预取的开始阶段不能大于等待完成的阶段。

$$\sum_{j=1}^n \sum_{1 \leq t \leq m} \sum_{m \leq k \leq n} O_{j,t,k} \leq 1, \quad 1 \leq m \leq n \quad (3.15)$$

$$\sum_{j=1}^n \sum_{1 \leq t \leq m} \sum_{m \leq k \leq n} P_{j,t,k} \leq 1, \quad 1 \leq m \leq n \quad (3.16)$$

$$O_{j,t,k} = 0, \quad j \geq t \vee j \geq k \vee k < t \quad (3.17)$$

$$P_{j,t,k} = 0, \quad j \geq t \vee j \geq k \vee k < t \quad (3.18)$$

卸载^[22,23,9,36]与计算是并行执行的, 在系统领域, 并行的操作容易造成读写的

错误, 必须严格保证卸载正确性。对于卸载, 当张量被卸载到 CPU 内存后, 内存里的张量可以作为备份。若被卸载的张量在后续计算中被多次访问, 则可以将内存中的张量多次预取回 GPU 的显存。公式 3.19 约束了对于任何节点的输出张量, 它只需要被卸载一次。对于显存中的张量, 卸载是读操作, 它必须保证被卸载的张量是可以访问的, 否则会传输脏数据。公式 3.20 定义了卸载开始时, 张量的数据是可以访问的, 而公式 3.21 定义了卸载结束时, 张量的数据也是可以访问的。

$$\sum_{t=1}^n \sum_{k=1}^n O_{j,t,k} \leq 1, \quad 1 \leq j \leq n \quad (3.19)$$

$$O_{j,t,k} \leq S_{t,j}, \quad 1 \leq j < t \leq k \leq n \quad (3.20)$$

$$O_{j,t,k} \leq S_{k,j}, \quad 1 \leq j < t \leq k \leq n \quad (3.21)$$

对于预取^[22,23,9,36], HGMOP 也必须保证预取行为的正确性, 避免造成并行的错误。首先, 预取不能传输未卸载过的张量, 否则会传输 CPU 内存中的脏数据到 GPU 显存。其次, 预取是读取 CPU 内存中的张量传输到 GPU 的显存, 而卸载将显存的张量写到 CPU 的内存。因此, 张量的预取需要在卸载完成之后开始, 否则卸载和预取会对内存中的相同地址同时读写, 这会导致程序的崩溃。公式 3.22 定义了如果没有预取过 j 计算节点的输出张量, 就不用卸载张量。而且, 一旦卸载过张量, 它可以多次被预取回显存。 α 是可以设置的整数参数, 它表示为张量的最大预取次数。公式 3.23 定义了 j 计算节点的输出张量的预取必须在卸载完成之后才能开始。若 $O_{j,t,k}$ 为 1, 则表示 j 计算节点的输出张量在 t 阶段开始卸载, 在 k 阶段等待卸载的完成。因此, 在 k 阶段之前, $P_{j,m,r}$ 必须为 0。

$$\sum_{t=1}^n \sum_{k=1}^n O_{j,t,k} \leq \sum_{t=1}^n \sum_{k=1}^n P_{j,t,k} \leq \alpha \sum_{t=1}^n \sum_{k=1}^n O_{j,t,k}, \quad 1 \leq j \leq n \quad (3.22)$$

$$O_{j,t,k} + P_{j,m,r} \leq 1, \quad 1 \leq j < t \leq k \leq n \wedge (1 \leq m \leq k \vee 1 \leq r \leq k) \quad (3.23)$$

3.3.3 张量的显存限制

为了避免执行计算节点时发生显存不足, HGMOP 定义了变量 U 表示执行节点的计算时 GPU 显存里张量占用的显存大小。 $U_{t,k}$ 定义为在 t 阶段计算 k 节点时张量占用了多少显存, 而 $U_{t,0}$ 定义了 t 阶段开始时占用的显存。如公式 3.24, 在每个阶段开始时, 显存中保留的张量是 DNN 的输入训练数据 M_{input} , 权重 M_{param} 和权

重梯度 M_{grad} ，以及在该阶段保持可访问的张量 $M_S(t)$ 。除此之外，若在 t 阶段预取某些计算节点的输出张量，它们也需要占用显存，即 $M_P(t)$ 。例如， $P_{j,l,r}$ 若为1，则 j 计算节点的输出张量会从 l 阶段开始预取，到 r 阶段等待预取的完成。若从 l 到 r 阶段的范围包括 t 阶段， $U_{t,0}$ 需要包括 j 计算节点的输出张量。

$$U_{t,0} = M_{input} + M_{param} + M_{grad} + M_S(t) + M_P(t), \quad 1 \leq t \leq n \quad (3.24)$$

$$M_S(t) = \sum_{j=1}^n S_{t,j} M_j, \quad 1 \leq t \leq n \quad (3.25)$$

$$M_P(t) = \sum_{j=1}^n \sum_{1 \leq l \leq t \leq r} P_{j,l,r} M_j, \quad 1 \leq t \leq n \quad (3.26)$$

在 t 阶段执行其它计算节点时，它们的显存占用可以通过递归的方式计算出来，即公式 3.27。在 t 阶段计算 $k+1$ 节点时，它的显存占用 $U_{t,k+1}$ 与 t 阶段计算 k 节点的显存占用 $U_{t,k}$ 相关。在 t 阶段计算 k 节点之后，它会释放部分 k 节点依赖张量的显存占用 $M_{free}(t,k)$ 。释放显存之后， $U_{t,k+1}$ 需要包括 $k+1$ 计算节点的输出张量的显存大小。

$$U_{t,k+1} = U_{t,k} - M_{free}(t,k) + R_{t,k+1} M_{k+1}, \quad 1 \leq t, k \leq n \quad (3.27)$$

$$M_{free}(t,k) = \sum_{i \in DEPS[k]} Free_{t,k,i} M_i, \quad 1 \leq t, k \leq n \quad (3.28)$$

$Free_{t,k,i}$ 定义为在 t 阶段计算 k 节点之后是否释放 i 计算节点的输出张量，它与很多因素相关，如公式 3.29 所示。首先， $(1 - S_{t+1,i})$ 表示了若 i 计算节点的输出张量在 $t+1$ 阶段不再继续保持可访问，即不在显存中保留张量，该张量的显存资源在 t 阶段计算 k 节点之后可以被释放。若 i 计算节点的输出张量继续保持可访问，该张量不能被释放。其次， $follow_s(t,k,i)$ 表示了若在 t 阶段计算 k 节点之后不再依赖 i 计算节点的输出张量，该张量在 k 节点执行之后可以被释放，否则不能被释放。 $USERS[i]$ 定义了依赖于 i 节点的其它计算节点。 $Free_{t,k,i}$ 中的 $R_{t,k}$ 变量避免了在 t 阶段时对同一个计算节点的输出张量进行重复释放。如果没有该变量，在 t 阶段时，若 i 计算节点的输出张量不再继续保持可访问且不在后续的计算中被依赖，则在 t 阶段的每个节点， $Free_{t,k,i}$ 都会为1，这会导致显存占用计算不正确。最后，公式 3.31 表示每个节点运行时的显存占用都不能超过显存容量的限制。 M_{gpu} 是可用的 GPU 显存容量大小， β 是可以设置的比例系数。

$$Free_{t,k,i} = R_{t,k} (1 - S_{t+1,i}) follow_s(t,k,i), \quad 1 \leq t, k, i \leq n \quad (3.29)$$

$$follow_s(t, k, i) = \prod_{j>k \wedge j \in USERS[i]} (1 - R_{t,j}), \quad 1 \leq t, k, i \leq n \quad (3.30)$$

$$USERS[i] = \{k: <v_i, v_k> \in E\}$$

$$U_{t,k} \leq \beta M_{gpu}, \quad 1 \leq t, k \leq n \quad (3.31)$$

然而，公式 3.29 是非线性的，会加重求解器寻找最优策略的负载。因此，有必要将其进行线性化。首先，线性化需要引入数学引理公式 3.32。在累乘公式中，任何一项出现了 0，整个累乘公式的结果为 0，而当在累乘公式中，所有项为 1 时，整个累乘公式的结果为 1。借助这个引理，公式 3.29 可以转化为线性的公式 3.33。其中， γ 是可以设置的正整数参数，它会影响搜索策略的速度和质量。

$$\prod_{i=1}^n x_i = \begin{cases} 1, & \sum_{i=1}^n (1 - x_i) = 0 \\ 0, & \text{其它} \end{cases}, \quad x_i \in \{0,1\} \quad (3.32)$$

$$lfree(t, k, i) = (1 - R_{t,k}) + S_{t+1,i} + lfollow_s(t, k, i)$$

$$lfollow_s(t, k, i) = \sum_{j>k \wedge j \in USERS[i]} R_{t,j}$$

$$(1 - Free_{t,k,i}) \leq lfree(t, k, i) \leq \gamma(1 - Free_{t,k,i}) \quad (3.33)$$

3.4 混合机制显存优化问题

定义 3-1：混合机制的显存优化问题是对于给定的 DNN 训练 $G = (V, E)$ ，HGMOP 在显存容量限制 M_{gpu} 下寻求高效的卸载和重计算结合策略，最小化显存优化代价。

$$\begin{aligned} \min_{R,S,FREE,O,P} & \sum_{i=1}^n \sum_{j=1 \wedge i \neq j}^n R_{i,j} C_j + \sum_{j=1}^n of o_j + \sum_{j=1}^n p f o_j \\ & \text{subject to (3.1)~(3.33)} \end{aligned} \quad (3.34)$$

该问题主要涉及了 R ， S ， O ， P ， $FREE$ 变量，它们分别为张量的计算，可访问性，卸载，预取和释放。为保证计算节点执行的正确性，公式(3.10)~(3.14)约束了计算节点依赖张量的正确性。为保证卸载和预取的正确性，公式(3.15)~(3.23)避免了发生读取脏数据，读写错误等等。除此之外，公式(3.24)~(3.33)保证了在训练时，每个节点执行时的显存占用不会超出可用的显存容量。最后，该问题以最小化卸载，预取和重计算的代价为目标，如公式 3.34 所示。对于有 n 个计算节点的数据

流图，混合机制显存优化问题主要涉及了 $2n^2 + 3n^3$ 个起主导作用的0或1整数变量，具有指数级规模的解空间（ $2^{2n^2+3n^3}$ ）。

3.5 算法设计

根据输入的数据流图和可用的显存容量，本章的混合机制显存优化问题可以利用求解器搜索出可行的策略。目标公式 3.34 中的 R ， S ， O ， P ， $FREE$ 变量都是0或1的变量，而 ofo 和 pfo 变量是非整数变量，它是一个混合整数规划问题。因此，本章利用 Gurobi^[41]求解器设计基于混合整数规划的显存优化算法 MIP-MO。

MIP-MO 算法产生一个显存管理调度 $L = \{l_1, l_2, l_3, \dots, l_m\}$ ， l_i 是显存管理的操作，如计算张量，释放张量，卸载张量等等。MIP-MO 的算法如算法 3.1 所示。MIP-MO 首先利用 Gurobi 搜索混合机制显存优化问题的 R ， S ， O ， P ， $FREE$ 变量。然后，它遍历所有的 R ， S ， O ， P ， $FREE$ 变量制定显存管理调度。在 i 阶段开始时，如果 $O_{x,i,y}$ 或 $P_{x,i,y}$ 是1，那么在 i 阶段，MIP-MO 要卸载或预取 x 节点的输出张量，添加显存管理操作卸载或预取张量到 L 中，即代码 5 到 7 行。如果 $R_{i,j}$ 是1，那么在 i 阶段，MIP-MO 要计算 j 节点的输出张量，添加显存管理操作计算张量到 L 中，即代码 11 到 13 行。如果 $Free_{i,j,t}$ 是1，那么在 i 阶段，执行 j 计算节点之后，MIP-MO 要释放 t 节点的输出张量，添加显存管理操作释放张量到 L 中，即代码 15 到 17 行。

算法 3.1: 基于混合整数规划的显存优化算法 MIP-MO

输入: 数据流图 $G = (V, E)$, 可用的显存容量 M_{gpu}

输出: 显存管理调度 $L = \{l_1, l_2, l_3, \dots, l_m\}$

1. 调用 Gurobi 求解器获得可行的策略 $R, S, O, P, FREE$
2. 循环: $i = 1, 2, \dots, |V|$ 执行:
 3. 循环: $x = 1, 2, \dots, |V|$ 执行:
 4. 循环: $y = 1, 2, \dots, |V|$ 执行:
 5. 如果 $O_{x,i,y} == 1$ 或 $P_{x,i,y} == 1$ 然后:
 6. 添加显存管理操作卸载或预取 x 节点的输出张量到 L 中
 7. 结束
 8. 结束
 9. 结束
10. 循环: $j = 1, 2, \dots, |V|$ 执行:
 11. 如果 $R_{i,j} == 1$ 然后:
 12. 添加显存管理操作计算 j 节点的输出张量到 L 中
 13. 结束
 14. 循环: t 张量是计算节点 j 的依赖张量 执行:
 15. 如果 $Free_{i,j,t} == 1$ 然后:
 16. 添加显存管理操作释放 t 节点的输出张量到 L 中
 17. 结束
 18. 结束
19. 结束
20. 结束

3.6 本章小结

本章首先对卸载和重计算不当结合的问题进行了详细描述。然后, 本章提出了混合显存优化统一代价模型。在此基础上, 本章添加显存管理约束, 提出了混合机制的显存优化问题 HGMOP。最后针对该问题, 本章设计了基于 Gurobi 求解器的显存优化算法 MIP-MO。

第4章 基于深度强化学习的显存优化算法

4.1 问题描述和算法概述

尽管 MIP-MO 算法能够获得高效的卸载^[22,23,9,36]和重计算^[26,10,13]结合策略,但在处理大规模 DNN 的显存优化问题时,该算法的时间成本相当昂贵。以 ResNet152^[20]的显存管理为例,它的训练数据流图具有 1080 个计算节点。MIP-MO 算法主要涉及的 0 或 1 整数变量约有 30 亿,这使得解空间极其庞大,即 0 或 1 整数变量组合的显存管理策略约有 $2^{(3 \times 10^9 + 2 \times 10^6)}$ 个。对于这种规模的解空间,整数规划的求解器难以在短时间内搜索到高质量的卸载和重计算的结合策略^[27,11]。此外,为了简化混合整数规划模型,MIP-MO 算法对卸载和预取进行了限制,例如,在每个阶段只有一个计算节点的输出张量进行卸载和预取。这一约束条件限制了对 PCI-E 总线资源的高效利用。针对这些不足,本章提出了基于深度强化学习的显存优化算法 DRL-MO。

强化学习进行决策的基本过程可以建模成一个马尔科夫决策过程。本文将 DNN 训练的显存管理规范化为离散的马尔科夫决策过程,如图 4.1 所示。环境的运行监视器收集 DNN 训练的信息,它包括显存占用情况,张量存活情况,运行反馈等信息。智能体以运行监视器观察到的显存状态作为输入,根据当前的策略,选择释放,卸载张量等动作。显存控制器根据智能体选择的动作释放,卸载,预取或计算张量。显存管理的决策过程可以通过三元组 (s, a, r) 来表达。 s 是环境的状态, a 是智能体对环境施加的动作, r 是环境被施加动作之后给智能体的反馈,即动作的执行时间,例如,执行一个计算节点的耗时。在学习显存管理策略的过程中,环境与智能体的交互会产生一个显存管理轨迹 τ ,如公式 4.1。 s_1 是一个训练迭代的初始状态。在该状态下,显存中只有权重张量,智能体只能选择一个动作 a_1 ,即执行数据流图的第一个计算节点。然后,显存中存在着第一个计算节点的输出张量,显存状态迁移为 s_2 。运行监视器反馈给智能体动作 a_1 的执行时间 r_1 ,即执行第一个计算节点的耗时,同时将显存当前的状态 s_2 反馈给智能体。智能体根据状态 s_2 选出下一个动作 a_2 。智能体与环境反复地进行上述过程的交互,直到终止状态 s_T 。然后,智能体根据收集到的显存管理轨迹 τ 来调整策略。

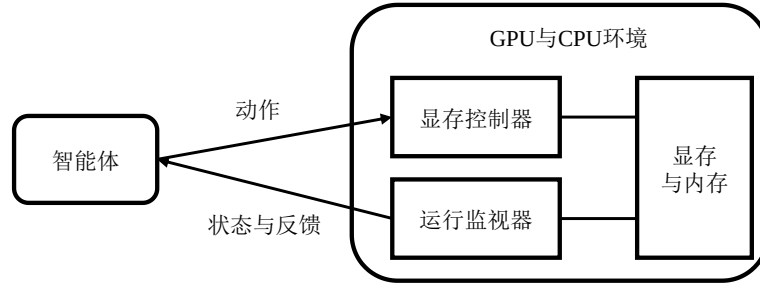


图 4.1 显存管理的强化学习基本过程

$$\tau = \{s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T\} \quad (4.1)$$

强化学习是智能体与环境不断地交互，以最大化奖励为目标，通过试错来学习最佳的策略。显存管理的目的是在有限的显存内，通过卸载^[22,23,9,36]和重计算^[26,10,13]的机制，从数据流图的第一个计算节点开始运行，直到最后一个计算节点完成所有权重梯度的计算，使得引入的卸载和重计算代价最小化。由于环境反馈给智能体动作的执行时间，所以，显存管理的强化学习的目标如公式 4.2。接下来，本章会从四部分详细介绍 DRL-MO 算法，它们分别为张量状态嵌入，张量动作空间设计，Actor 和 Critic 网络设计以及训练算法。

$$\text{maximize} \sum_{t=1}^T -r_t \quad (4.2)$$

4.2 张量状态嵌入

训练数据流图以混合机制显存优化问题的 $G = (V, E)$ 来表述，而张量之间的状态会相互影响，例如，某个张量的依赖张量被释放，它的重计算时间就会增加。因此，DRL-MO 有必要提取张量之间的拓扑结构联系。图神经网络^[51,52] (Graph Neural Network, GNN) 是对图数据进行处理的一类神经网络，它可以提取图数据中的拓扑结构信息。为了提升 GNN 提取特征的能力，研究人员提出了许多改进的图神经网络，如图卷积神经网络^[51] (Graph Convolutional Network, GCN)，图同构神经网络^[52] (Graph Isomorphism Network, GIN) 等。由于 GIN^[52] 强大的信息提取能力，DRL-MO 利用 GIN 来提取计算节点之间拓扑结构信息的特征。

在环境中执行显存管理动作后，涉及的计算节点的输出张量状态会随之改变，导致整个显存系统的状态发生变化。在每个 GPU 环境状态下，这些张量状态对智能体进行显存管理决策产生影响。DRL-MO 将 s_t^i 定义为在 s_t 状态下 i 计算节点的输

出张量的状态信息。它包括 GPU 的空闲显存大小 GM_t ，张量的显存占用大小 M_i ， i 计算节点的运行耗时 C_i ，卸载耗时 CO_i ，预取耗时 CP_i ，是否占用显存 E_t^i ，是否已在 CPU 内存备份 BC_t^i ，剩余卸载时间 LCO_t^i ，剩余预取时间 LCP_t^i 和重计算时间 RC_t^i 。 s_t 包含了当前状态下所有的张量状态信息。

$$s_t^i = (GM_t, M_i, C_i, CO_i, CP_i, E_t^i, BC_t^i, LCO_t^i, LCP_t^i, RC_t^i) \quad (4.3)$$

$$s_t = (s_t^1, s_t^2, \dots, s_t^n) \quad (4.4)$$

GIN 以 s_t 作为输入，通过公式 4.5 迭代地提取张量之间的拓扑结构信息。 $h_{t,i}^{(k)}$ 是在状态 s_t ，GIN^[52] 经过 k 次迭代提取的 i 计算节点输出张量的状态特征信息。注意，GIN 的原始输入 $h_{t,i}^{(0)}$ ，即为 s_t^i 。其中， $MLP_{\theta_k}^{(k)}$ 是 GIN 第 k 次迭代提取特征时利用的多层感知机 (Multi-Layer Perceptron, MLP)， θ_k 是多层感知机对应的权重， ϵ 是可学习的参数。 $\mathcal{N}(i)$ 是 i 节点相邻的计算节点集合。虽然 GIN 是针对无向图设计的图神经网络，但仍然适合于基于 DAG 的训练数据流图。因为一个计算节点的输出张量既会影响它的后继计算节点，也会影响依赖它的前驱计算节点。例如，在图 3.1 中，若 v_3 计算节点的输出张量被释放，则会影响其前驱计算节点 v_2 的重计算次数。因为若 v_2 计算节点的输出张量也被释放，则重新执行 v_3 计算节点时需要重计算 v_2 节点。公式 4.5 提取了每个计算节点输出张量的状态特征。但为了更全面地选择动作，有必要知道整个显存的状态信息。 $h_{t,G}^{(K)}$ 是在状态 s_t ，GIN 经过 K 次迭代提取的全局状态信息。它通过平均所有张量的状态特征信息来获得全局状态信息。

$$h_{t,i}^{(k)} = MLP_{\theta_k}^{(k)} \left((1 + \epsilon^k) h_{t,i}^{(k-1)} + \sum_{v \in \mathcal{N}(i)} h_{t,v}^{(k-1)} \right) \quad (4.5)$$

$$h_{t,G}^{(K)} = \frac{\sum_{i \in V} h_{t,i}^{(K)}}{|V|} \quad (4.6)$$

4.3 张量动作空间设计

在每个状态下，计算节点的输出张量都可以进行四种动作，分别为计算，卸载，预取和释放。智能体在选择动作时，它应该考虑 GPU 全局的状态信息 $h_{t,G}^{(K)}$ 和每个计算节点输出张量的状态信息 $h_{t,i}^{(K)}$ 。此外，每个动作对显存的占用和训练迭代时间都会造成影响。例如，执行某个节点的计算的动作，它会增加显存的占用，同

时也会延长训练的迭代时间。DRL-MO 的每个动作 a_t^i 定义为在状态 s_t , i 计算节点输出张量的动作特征信息, 如公式 4.7。其中, a 是动作的类型, 它们分别为计算 (c), 卸载 (o), 预取 (p) 和释放 (f)。 FM_i 是 i 计算节点输出张量的动作执行之后 GPU 的空闲显存大小, OC_i 是执行该动作耗费的时间。

$$a_t^i = (FM_i, OC_i, h_{t,i}^{(K)}, h_{t,G}^{(K)}) \quad (4.7)$$

为了确保数据流图的顺序执行, 即从第一个节点依序执行到最后一个节点, DRL-MO 采用了计算序列 (CompSeq) 的机制。计算序列是计算节点执行的序列, 其中的元素表示需要执行的计算节点, 序列的前端表示下一个要执行的计算节点。在数据流图开始执行时, 计算序列的元素依序是第一个计算节点到最后一个计算节点。随着训练的进行, 由于张量的释放与再生, 计算序列需要根据 GPU 显存中的张量状态进行更新。DRL-MO 设计了一种基于递归的计算序列更新算法, 如算法 4.1 所示。该算法检查下一个计算节点依赖张量的状态。如果张量正在被卸载、预取或者在显存中, 则无需进行重计算, 反之, 如果张量不存在于显存中, 则将该张量添加到计算序列的前端, 以确保进行重计算。然而, 进行重计算的计算节点必须保证计算的正确性, 因此, 算法会继续调用更新序列的过程。通过这种方式, DRL-MO 有效地维护了一个适应动态显存状态的计算序列, 保证了计算节点执行的有序性。

算法 4.1: 计算序列更新算法

输入: 现有的计算序列 CompSeq, 下一个计算节点 c , c 计算节点依赖的节点 $DEPS[c]$

输出: 更新后的计算序列 CompSeq

1. 循环: t 属于 $DEPS[c]$ 执行:
 2. 如果 t 计算节点的输出张量不在显存中, 然后:
 3. 将 t 计算节点插入到 CompSeq 的前端
 4. 根据计算序列 CompSeq, 下一个计算节点 t 以及其依赖节点 $DEPS[t]$, 调用计算
 5. 序列更新算法
 6. 结束
 7. 结束
-

此外, 在显存管理的调度中, 施加于环境的动作必须满足一系列约束条件, 以确保其正确性。例如, 预取动作必须在卸载完成之后才能开始。为了保证动作的正确性, DRL-MO 引入了动作过滤算法, 通过该算法挑选出满足约束条件的可行动作, 具体实现如算法 4.2 所示。首先, 算法根据当前的张量状态更新计算序列

CompSeq, 其前端元素是下一个要执行的计算节点。但执行计算动作会增加显存的占用, 若空闲显存不满足计算动作的显存需求, 可行的动作集合就不会包括执行该节点计算的动作。对于卸载动作, 不是所有计算节点的输出张量都可以被卸载, 因为卸载必须满足两个约束条件: 首先, 只有可访问的张量才能被卸载; 其次, 已经被卸载的张量不能再次被卸载, 因为 CPU 内存中存在该张量的备份。类似于卸载, 只有符合约束条件的张量才能进行预取操作: 预取的张量必须是之前已被卸载过的, 因为张量只有在被卸载后, CPU 内存中才存在该张量的备份; 预取的张量不能存在于显存中, 以避免浪费 PCI-E 总线资源; 同时, 预取的张量不能超出 GPU 显存的可用容量。释放动作也需要满足一些约束条件, 包括: 张量必须存在于显存中, 且不是可行的计算动作的依赖张量。通过这样的动作过滤机制, DRL-MO 确保了显存管理动作的有效性。

算法 4.2: 动作过滤算法

输入: 空闲显存大小 U , 计算节点的数量 n

输出: 可行的动作集合 FeaActions

1. 初始化可行动作集合 FeaActions 为空集, 更新计算序列 CompSeq
 2. 取出计算序列 CompSeq 中下一个要执行的计算节点, 作为计算动作 c
 3. 如果计算动作 c 所需显存 $\leq$ 空闲显存大小 U , 然后:
 4. 作为可行的计算动作 c , 加入 FeaActions 中
 5. 结束
 6. 循环: $t = 1, 2, \dots, n$ 执行:
 7. 如果 t 计算节点的输出张量在显存中, 且没有被卸载过, 然后:
 8. 作为可行的卸载动作 o , 加入 FeaActions 中
 9. 结束
 10. 如果 t 计算节点的输出张量已经被卸载, 没存在显存中, 且张量显存大小 $\leq$ 空闲显存大小 U , 然后:
 12. 作为可行的预取动作 p , 加入 FeaActions 中
 13. 结束
 14. 如果 t 计算节点的输出张量在显存中, 不是计算 c 的依赖张量, 然后:
 15. 作为可行的释放动作 f , 加入 FeaActions 中
 16. 结束
 17. 结束
-

4.4 Actor 与 Critic 网络设计

在每个状态下存在着多个动作，DRL-MO 需要选择合适的动作施加到 GPU 环境，以最小化训练迭代时间。为此，DRL-MO 设计了基于 MLP 的 Actor 神经网络。每个动作都是由特征信息组成，Actor 神经网络通过特征信息来预测动作的概率分布，如公式 4.8。其中， p_t 是状态 s_t 下所有可行动作被选择的概率分布， a^t 是所有可行动作的特征信息， θ_{Actor} 是 Actor 神经网络的权重。首先，Actor 网络根据动作特征信息评估动作的优劣值，然后通过 *softmax* 函数将优劣值转化为概率分布。

$$p_t = \text{softmax}(MLP_{\theta_{Actor}}(a^t)) \quad (4.8)$$

为了使得 Actor 神经网络更好地收敛，DRL-MO 设计了基于 MLP 的 Critic 网络。Critic 网络负责预测某状态下选择的动作能获得的最小训练迭代时间。因此，它的输入是状态嵌入网络的全局状态特征向量 $h_{t,G}^{(K)}$ 和选择的动作特征向量 a_t ，如公式 4.9。 θ_{Critic} 是 Critic 网络的权重。

$$Q^{\pi_{\theta}}(s_t, a_t) = MLP_{\theta_{Critic}}(h_{t,G}^{(K)}, a_t) \quad (4.9)$$

4.5 优势演员-评论家训练算法

为了寻找最佳的 DRL-MO 的权重，本文使用了优势演员-评论家训练算法^[47]。传统的策略梯度方法使用公式 4.10 计算权重梯度来使得期望训练迭代时间最小化。 θ 是 DRL-MO 的权重，它包括状态嵌入的网络权重，Actor 网络权重和 Critic 网络权重。 a_t 是在状态 s_t 时选择的动作。

$$\nabla E_{\pi_{\theta}} \left[\sum_{t=1}^T -r_t \right] = E_{\pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(s_t, a_t) \left(\sum_{t=1}^T -r_t \right) \right] \quad (4.10)$$

传统的策略梯度算法受到梯度方差较大的影响，导致训练过程不够稳定。该不稳定性主要源于策略梯度法未能有效考虑动作之间的优劣差异性。具体而言，对于一个给定的轨迹，存在既有好的动作又有坏的动作的情况。然而，由于累积奖励是正反馈时，策略梯度法倾向于将整个轨迹中的所有动作视为好动作，从而盲目地增加所有动作的选择概率。反之，当累积奖励为负反馈时，策略梯度法倾向于将整个轨迹中的所有动作都视为坏动作，导致盲目地降低所有动作的选择概率。这种行

为使得训练过程变得不够稳定。

算法 4.3: 模拟运行算法

输入: 选择的动作 a , 依赖计算节点 $DEPS$

输出: 可行的动作 $FeaActions$, 所有张量的状态信息 s , 动作执行时间 r

1. 如果动作 a 是释放动作, 然后:
 2. a 选择了释放 t 计算节点的输出张量
 3. 如果 t 计算节点的输出张量处于卸载或预取状态, 然后:
 4. 等待张量完成卸载或预取, 张量状态转为在显存中
 5. 结束
 6. 如果 t 计算节点的输出张量存在于显存中, 然后:
 7. 释放显存资源, t 计算节点的输出张量状态转为不存在于显存中
 8. 结束
 9. 结束
 10. 如果动作 a 是计算动作, 然后:
 11. 执行的计算节点是 j , 其依赖计算节点为 $DEPS[j]$
 12. 循环: t 属于 $DEPS[j]$ 执行:
 13. 如果 t 计算节点的输出张量处于预取状态, 然后:
 14. 等待张量预取完成, 张量状态转为在显存中
 15. 结束
 16. 结束
 17. 为 j 计算节点的输出张量申请显存
 18. 执行 j 计算节点, j 计算节点的输出张量状态转为在显存中
 19. 结束
 20. 如果动作 a 是预取动作, 然后:
 21. a 选择了预取 t 计算节点的输出张量
 22. 为张量申请显存, 将张量状态转为在预取
 23. 结束
 24. 如果动作 a 是卸载动作, 然后:
 25. a 选择了卸载 t 计算节点的输出张量, 将张量状态转为卸载
 26. 结束
 27. 收集所有张量的状态信息 s 和动作执行时间 r
 28. 根据可用显存大小 U , 计算节点数量 n , 调用动作过滤算法获得可行动作集合 $FeaActions$
-

优势演员-评论家训练算法^[47]引入了优势函数 A 来量化选择动作的优劣, 使得强化学习的训练感知到动作优劣的差异性, 从而提升轨迹里好的动作被选择概率,

减少坏的动作被选择概率，如公式 4.11 所示。 G_t 是从环境收集的从 s_t 到终止状态的累积运行时间， $Q^{\pi_{\theta}}(s_t, a_t)$ 是 Critic 网络预测的在 s_t 选择 a_t 动作，直到终止状态的最短运行时间。

$$A^{\pi_{\theta}}(s_t, a_t) = G_t - Q^{\pi_{\theta}}(s_t, a_t) \quad (4.11)$$

所以，优势演员-评论家算法的 Actor 网络的权重梯度计算为公式 4.12。

$$\sum_t \nabla_{\theta_{Actor}} \log \pi_{\theta}(s_t, a_t) A^{\pi_{\theta}}(s_t, a_t) \quad (4.12)$$

Critic 网络预测某个状态下选择某动作，直到终止状态的训练迭代时间，它的权重梯度计算为公式 4.13。

$$\sum_t \nabla_{\theta_{Critic}} (G_t - Q^{\pi_{\theta}}(s_t, a_t))^2 \quad (4.13)$$

DRL-MO 通过与环境的交互来调整策略，但由于通过真实训练收集状态信息存在时间成本昂贵的问题，因此，采用模拟器来有效地收集状态信息。为此，DRL-MO 引入了模拟运行算法，如算法 4.3 所示。在该算法中，若所选动作是释放张量，则会释放相应的显存资源。如果选择的张量正在卸载或预取，则需要等待卸载或预取过程的完成。若所选动作是执行计算节点，则会为计算节点的输出张量申请显存资源。如果计算节点的依赖张量正在预取，则必须等待其预取过程完成。若所选动作是预取张量，则算法会为相关张量申请显存资源并执行其预取操作。如果选择卸载张量的动作，则会执行相应的卸载操作。

DRL-MO 的优势演员-评论家训练算法^[47]如算法 4.4 所示。在每次迭代时，状态嵌入网络基于环境模拟运行算法反馈的状态信息提取每个张量的状态特征和显存的全局状态特征。DRL-MO 通过组合张量的状态特征、显存的全局状态特征以及可行动作的信息，构建动作的特征向量，通过 Actor 网络预测可行动作的概率分布。由于强化学习需要通过试错来学习显存管理策略，DRL-MO 使用概率分布来选择动作。例如，在某个状态下存在两种可行的动作 x 和 y ，它们被选择的概率分别是 80%和 20%，则有 80%的概率选择动作 x ，20%的概率选择动作 y 。将选择的动作应用于环境，通过模拟运行获取动作的执行时间反馈，以及更新后的张量状态信息和可行的动作集合，从而继续与环境进行交互。直到终止状态，DRL-MO 收集到一个从初始状态到终止状态的轨迹信息。从轨迹信息中随机选择一批状态、动作和反馈信息，DRL-MO 利用此数据计算状态嵌入网络，Actor 和 Critic 网络的权重梯度，更新它们的权重。然后，进入下一个迭代，直至完成所有的迭代。

算法 4.4: 基于优势演员-评论家的训练算法

输入: 迭代次数 I , 数据流图 G , 空闲显存大小 U

输出: 状态嵌入, Actor 和 Critic 网络的权重

1. 初始化状态嵌入, Actor 和 Critic 网络的权重
 2. 循环: $i = 1, 2, 3, \dots, I$ 执行:
 3. 循环: True 执行:
 4. 根据张量状态信息, 状态嵌入网络给出张量状态特征, 全局状态特征
 5. 根据张量状态特征, 全局状态特征, 以及可行的动作信息, 通过 Actor 网络计
 6. 算动作的概率分布
 7. 根据动作的概率分布选择动作
 8. 施加选择的动作到环境上, 通过模拟运行, 获得动作的执行时间反馈, 当前
 9. 的张量状态信息, 可行的动作集合
 10. 如果达到终止状态, 然后:
 11. 跳出循环
 12. 结束
 13. 结束
 14. 根据轨迹信息, 计算 Actor 网络和 Critic 网络的损失函数
 15. 根据损失函数, 计算权重梯度, 更新权重
 16. 结束
-

4.6 本章小结

本章首先描述了面向大规模显存优化问题时, MIP-MO 算法存在时间成本昂贵的不足, 提出并概述了基于深度强化学习的显存优化算法 DRL-MO。然后, 本章介绍了该算法的四个组成: 张量状态嵌入, 张量动作空间设计, Actor 和 Critic 网络设计以及优势演员-评论家训练算法。

第5章 训练平台和实验结果分析

5.1 Ant-NN 训练平台

现有训练框架未能提供完善的接口，在卸载和重计算机制的实现方面存在一定的限制。本文基于 C++，CUDA 和 CUDNN^[53]实现了轻量的深度神经网络训练平台 Ant-NN。该平台面向 DNN 的研究人员，透明地为训练进行显存优化。为了方便管理显存，Ant-NN 实现了卸载/预取和重计算等灵活的张量管理机制。而且该平台支持各种类型的 DNN，包括卷积，注意力等等，并提供了拓展接口，以支持新型 DNN 和显存优化算法的集成。通过这些接口，Ant-NN 集成了上述的两种显存优化算法 MIP-MO 和 DRL-MO，实现了从全卸载、DRL-MO 到 MIP-MO 渐进式显存优化策略调整。如图 5.1 所示，Ant-NN 平台由五大模块组成：神经网络库模块，数据流图提取模块，数据流图测量模块，显存策略制定模块和执行模块。

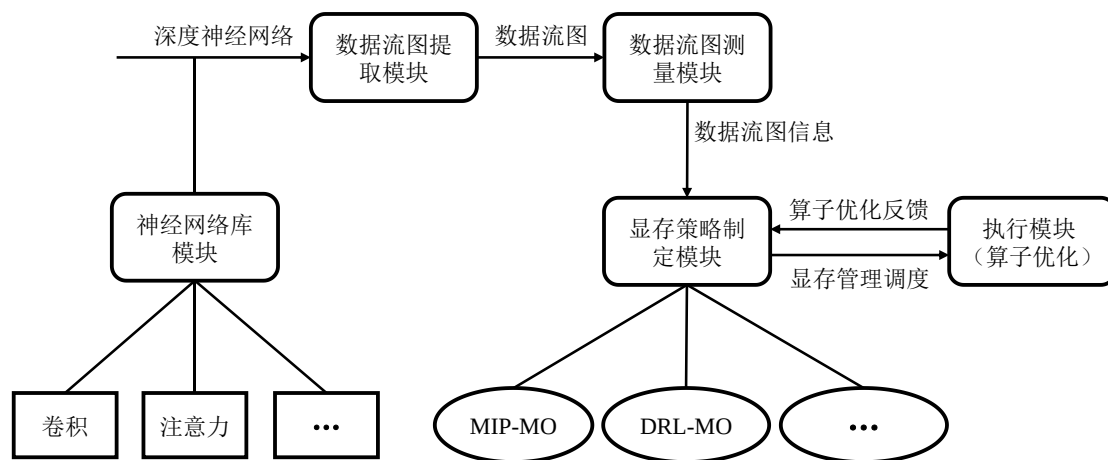


图 5.1 Ant-NN 平台

神经网络库模块实现了主流的隐藏层，研究人员可以利用这些隐藏层设计 DNN 的架构。数据流图提取模块将 DNN 的训练抽象为更通用的数据流图。数据流图测量模块根据数据流图测量每个计算节点执行时间，以及依赖张量的显存大小。显存策略制定模块利用测量模块提供的数据流图信息，在有限的显存容量限制下，通过优化算法制定显存管理策略。执行模块利用显存策略制定模块的显存管理调度来完成训练。为了充分利用 GPU 计算资源与 PCI-E 总线资源的并行性，执行模块采用三个 CUDA 流，分别负责计算张量、卸载张量和预取张量。

5.1.1 Ant-NN 训练平台特性

张量灵活管理机制：Ant-NN 基于张量粒度管理显存，并为了支持多种显存管理策略，设计了 8 种灵活的张量操作。这些操作包括：张量计算，计算等待，张量卸载，张量预取，卸载等待，预取等待，张量分配和张量释放。张量的计算操作是向计算流中提交执行计算的任务，而等待计算是让 CPU 等待计算流中张量计算的任务。卸载操作是向卸载流中提交张量传输的任务，而等待卸载是让 CPU 等待卸载流中传输任务的完成。预取相关的操作与卸载类似。张量的分配和释放操作是为其申请和释放 GPU 的显存资源。

平台拓展性：为了支持新型的 DNN，Ant-NN 的神经网络库模块提供了前向传播和后向传播接口。通过为新类型的隐藏层实现这两种接口，该平台就可以支持新型的 DNN。此外，Ant-NN 平台也支持添加新的显存优化算法。但不同的优化算法对卸载和重计算的显存管理策略具有不同的表述，如 MIP-MO 的表述就是卸载，预取和重计算等变量的 0 或 1 值。为了在执行模块中实现对基于不同表述策略的运行，Ant-NN 平台设计了统一的显存管理调度 $L = \{l_1, l_2, l_3, \dots, l_m\}$ ， l_i 是上述的张量操作。不同显存优化算法需要实现策略制定模块的接口，将显存管理策略转化为统一的显存管理调度，例如，MIP-MO 算法的伪代码，但它需要添加一些张量操作，如在提交张量计算之后添加等待计算的操作。等待计算是为了保证没有正在执行的计算节点依赖显存里的张量，以防止对张量同时进行读写。DRL-MO 在推理时可以通过收集模拟运行算法的调度来获得统一的显存管理调度。因为模拟运行算法模拟了训练中的张量计算，卸载，等待卸载等所有的张量操作。

算子优化：目前，张量计算涉及多种性能不同的算子实现^[54,55]，这些实现方式具有各自的显存需求和运行耗时。然而，若训练平台在选择算子实现时盲目追求耗时短，这可能会显著增加显存占用，导致显存不足的问题。Ant-NN 默认为计算节点配置显存需求较小的算子实现，以避免造成显存不足。在显存优化实验中，本文观察到 GPU 在某些时间段内存在空闲显存，这为执行模块提供了进行算子选择的机会。基于此发现，Ant-NN 在执行模块中引入了算子优化模块，该模块能够根据空闲显存的大小自动选择运行耗时最短的算子实现。如果算子优化模块能够提供更好的性能表现，它将反馈算子信息给显存策略制定模块，以供制定新的显存优化策略。这种机制有助于在训练过程中灵活地选择适当的算子实现，从而更充分地

利用显存资源。

5.1.2 渐进式显存优化策略调整

当可用显存容量无法满足 DNN 训练显存需求时,显存策略制定模块通过渐进式显存优化策略调整来选择适当的显存管理策略,如图 5.2 所示。首先,为了迅速响应用户需求,该模块提供一个全卸载策略,即将所有张量卸载到 CPU 内存。在训练过程中,显存策略制定模块利用 DRL-MO 算法制定新的显存管理策略。如果新策略的性能优于全卸载,执行模块则替换为 DRL-MO 算法的显存管理。接着,显存策略制定模块利用 MIP-MO 算法搜索更优的策略。如果 MIP-MO 算法在 3 小时内找到更优的策略,执行模块则将当前显存管理替换为 MIP-MO 算法的。通过渐进式显存优化策略调整, Ant-NN 在保证快速响应的同时,不断优化显存管理策略保证用户对于性能的需求。

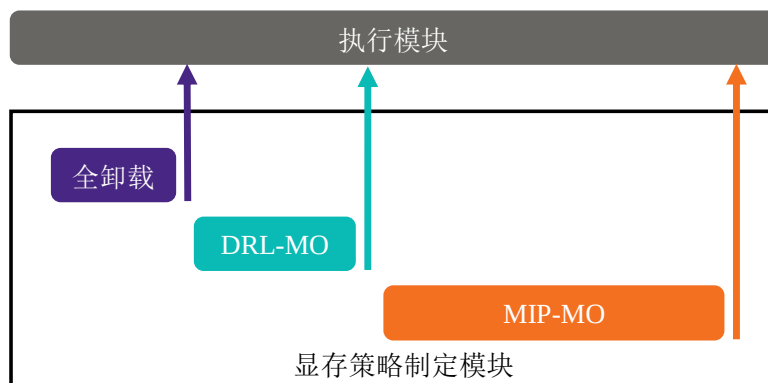


图 5.2 渐进式显存优化策略调整示例

5.2 实验环境与对比用例参数

对于实验,本文选取了多个经典的深度卷积神经网络,它们代表了主流的网络架构,表 5.1 展示了它们训练的相关信息。VGG19^[19]和 UNet^[56]是小规模 DNN,它们具有几十个隐藏层,训练数据流图具有低于 200 个计算节点。而 Inception V4^[21]和 ResNet152^[20]是大规模 DNN,它们有百层的隐藏层,训练数据流图拥有约 1000 个计算节点。VGG19, Inception V4 和 ResNet152 常用于图像识别,它们训练使用的数据集是 ImageNet^[18],图片大小是 227x227。UNet 常用于图像分割,它使用的数据集是 Data Science Bowl,图片大小是 256x256。

表 5.1 实验的深度神经网络

深度神经网络	训练数据集	训练数据流图节点数量
VGG19 ^[19]	ImageNet	118
UNet ^[56]	Data Science Bowl	156
Inception V4 ^[21]	ImageNet	999
ResNet152 ^[20]	ImageNet	1080

Ant-NN 是基于 CUDA 和 CUDNN 实现的。CUDNN^[53]是英伟达的 DNN 加速库，它实现了训练常用的张量计算，例如卷积，池化，和激活等等，它是主流训练框架的基础。Ant-NN 系统使用的 CUDA 和 CUDNN 的版本分别是 12.0 和 8.8.0。对于 MIP-MO 算法的求解器，Ant-NN 使用的 Gurobi^[41]版本是 10.0，算法的运行时限设置为 3 小时。对于 DRL-MO 算法，Ant-NN 使用版本为 1.10 的 Pytorch^[50]进行强化学习训练，以 VGG16^[19]和 UNet^[56]显存不足的问题作为训练集。实验环境的服务器采用了 Ubuntu18.04 操作系统，CPU 是 Intel(R) Xeon(R) CPU E5-2620 v4，内存容量是 62GB，使用的 GPU 为英伟达的 Tesla P40，显存容量是 24GB，而 PCI-E 总线的卸载和预取速度分别达到 12GB/s 和 11GB/s。

本文选取了四种显存优化策略作为基准，分别为 LMS^[9], DTR^[13], Checkmate^[10] 和 Capuchin^[11]。在 Ant-NN 平台上，本文通过增加训练的批次大小，增加训练的显存需求，测试 DNN 的训练吞吐量。为了避免显存碎片化的影响，显存优化算法的显存容量限制为 0.6 比例的可用显存容量。

LMS: 只利用卸载来降低训练的显存需求。LMS 设置张量的产生访问间距会过滤掉不适合卸载和预取的张量。为了减少预取阻塞时间，LMS 设置了卸载张量的预取提前距离，并通过设置预取共享距离减少预取次数。对于这三种距离参数，LMS 设计了基于二分查找的搜索算法。

DTR 和 Checkmate: 只利用重计算来降低训练的显存需求。为了减少重计算的额外时间，DTR 设计了选择被重计算的启发式量化标准。该标准将计算时间短，显存占用大，未被访问时间长的张量作为重计算目标。Checkmate 将重计算代价的最小化问题抽象为整数规划问题，并利用现有的 Gurobi^[41]求解器搜索重计算的策略。但对于大规模显存优化问题，Checkmate 面临搜索时间成本昂贵的困境。

Capuchin: 通过卸载和重计算的结合来降低训练的显存需求。Capuchin 定义了显存优化代价量化标准，利用它们估计张量的卸载，预取和重计算代价来搜索结

合策略。若张量的预取代价小于重计算代价，则张量需要被卸载和预取，否则张量需要被重计算。

5.3 实验结果与分析

5.3.1 MIP-MO 的结果与分析

由于 VGG19^[19]和 UNet^[56]具有更小的网络规模，本文通过它们来验证 MIP-MO 算法的性能，并选取了 LMS^[9]，Capuchin^[11]和 Checkmate^[10]算法作为对比。

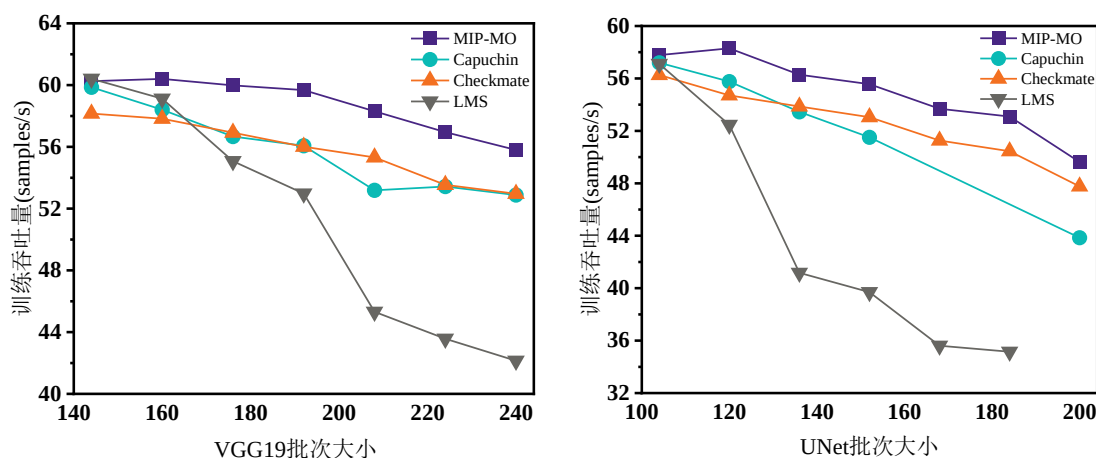


图 5.3 不同训练批次的 VGG19 和 UNet 训练吞吐量

与其它算法相比，MIP-MO 算法在显存需求较大时实现了更高的训练吞吐量。从图 5.3 所知，在小批次训练，显存需求较小时，这 4 种显存优化算法取得了相似的性能。然而，随着批次的增加，显存需求变大，这些算法出现了不同程度的训练性能恶化。在批次较小时，LMS 通过卸载和预取获得了高的训练吞吐量。然后，随着批次的增加，LMS 必须卸载和预取更多的张量，引入了更多的阻塞时间，导致训练性能迅速恶化。在批次较小时，Capuchin 和 Checkmate 的训练吞吐量相近。但随着批次的增加，Capuchin 由于未充分考虑两种机制的相互影响，产生了低效的结合策略，训练性能逐渐弱于 Checkmate，甚至导致显存不足。例如，在批次大小为 160 的 UNet 网络训练时，Capuchin 在执行 132 号计算节点时需要重计算 20 号节点，而 20 号节点依赖于 19 号节点的输出张量。然而，Capuchin 考虑了执行 132 号节点依赖的 20 号计算节点的显存，却未考虑重计算 19 号节点时需要的显存资源，出现了显存不足。而 MIP-MO 利用精确的混合整数规划模型计算出节点运行时的显存占用以及显存优化代价，可以获得高效的卸载和重计算的结合策略。随

随着显存需求增加，MIP-MO 的训练吞吐量普遍高于其它的算法。相比 Capuchin，Checkmate 和 LMS 算法，MIP-MO 训练吞吐量分别达到 13.1%，6.5%，51%的提升，平均情况下达到 23.5%的提升。

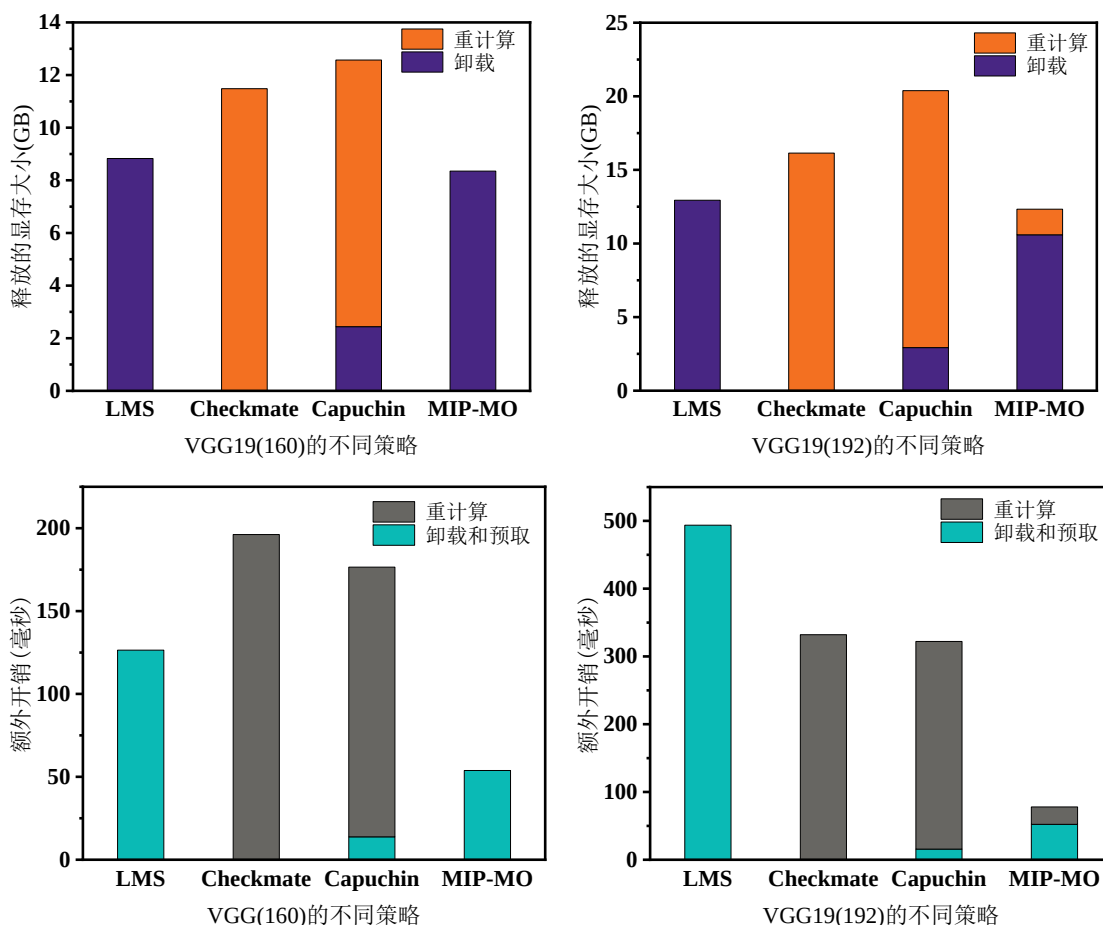


图 5.4 不同策略的释放显存和额外开销分析

为了探索 MIP-MO 的性能提升的原因，本文进一步分析了这 4 种显存优化算法。图 5.4 是 VGG19^[19]在批次 160 和 192 训练时，不同显存管理策略释放的显存资源和引入的额外开销。在批次 160 的训练时，LMS^[9]的训练吞吐量普遍高于 Capuchin 和 Checkmate^[10]。这是因为 LMS 充分利用卸载和预取减少了性能损失，而 Capuchin^[11]仅通过卸载释放了 2.44GB 的显存，增加了重计算的开销。但随着批次的增加，在批次 192 时，LMS 的训练吞吐量低于其它的算法。这是因为 LMS 引入了更多的卸载和预取的开销，而 Capuchin 利用重计算减少了卸载和预取导致的性能损失。但 Capuchin 不当的结合策略导致了训练吞吐量低于 MIP-MO。

MIP-MO 性能提升的主要原因是在不同的显存需求下，通过不同程度的卸载和预取减少重计算的开销。例如，在批次大小为 160 的 VGG19^[19]训练时，对于释放的显存资源，MIP-MO 的卸载占据了 100%，它只有 53.84 毫秒的卸载和预取开

销。而 Capuchin^[11]的卸载仅占据了 19.42%，重计算占据了 80.58%，它引入了 176.4 毫秒的额外开销，其中，卸载和预取开销是 13.7 毫秒，而重计算开销是 162.7 毫秒。而在批次大小为 192 时，显存需求进一步增加，MIP-MO 调整了卸载和重计算的结合。对于释放的显存资源，MIP-MO 的卸载占比降低至 85.87%，重计算增加到 14.13%。它引入了 77.9 毫秒的额外开销，其中，卸载和预取的开销是 52.3 毫秒，重计算开销是 25.6 毫秒。而 Capuchin 的卸载仅占据 14.39%，重计算占据了 85.61%。它引入了 322 毫秒的额外开销，其中，卸载和预取的开销是 15.7 毫秒，重计算开销是 306.3 毫秒。因此，MIP-MO 充分利用了卸载和预取来减少重计算的开销。

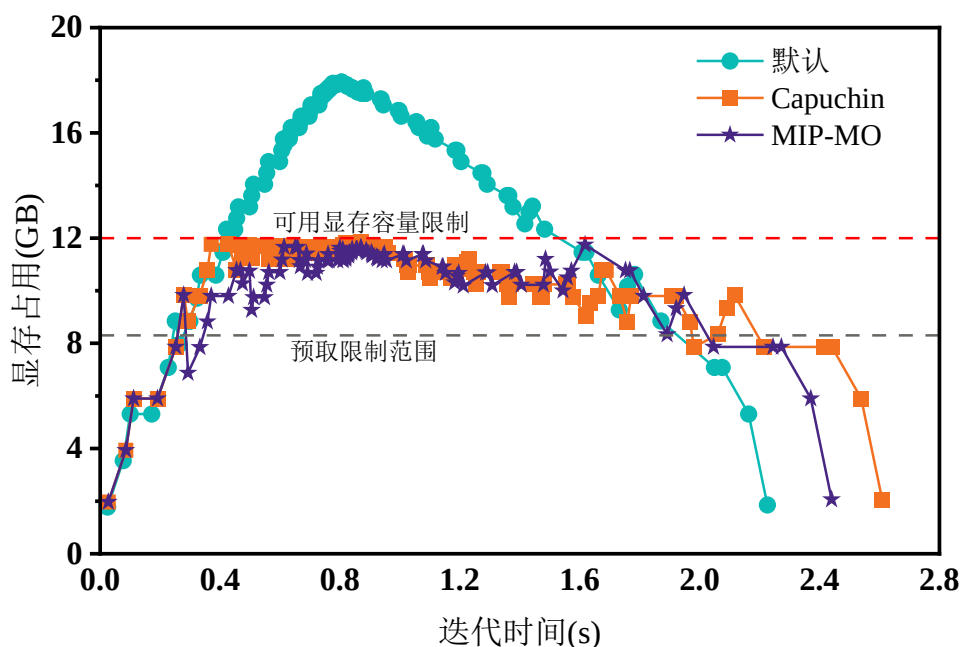


图 5.5 Capuchin 和 MIP-MO 的显存占用

为了探究卸载和预取的利用比例不同，本文分析了 Capuchin^[11]和 MIP-MO 的显存管理调度。图 5.5 是 VGG19 在批次为 160 训练时，MIP-MO 和 Capuchin 优化策略的显存占用分析，默认是不使用优化方法的显存占用曲线图，预取限制范围是 Capuchin 认为无代价预取张量的范围。相比于 MIP-MO，Capuchin 未充分考虑两种机制的相互影响，这阻碍了对卸载和预取机制的利用。为了防止预取造成的显存不足，Capuchin 限制了预取的范围，并认为该范围内的预取是无代价的。例如，在图 5.5 中，它在显存负载低于 8.3GB 之后才开始预取张量。然而，由于引入了重计算，这减少了显存占用且延后了张量的访问。因此，在显存的容量限制条件下，无代价或低代价的预取并不局限于该范围内。而不同于 Capuchin，MIP-MO 利用混

合机制显存优化问题模型充分地考虑了两种机制的结合，在显存负载低于 11.4GB 之后便开始预取张量。

5.3.2 DRL-MO 的结果与分析

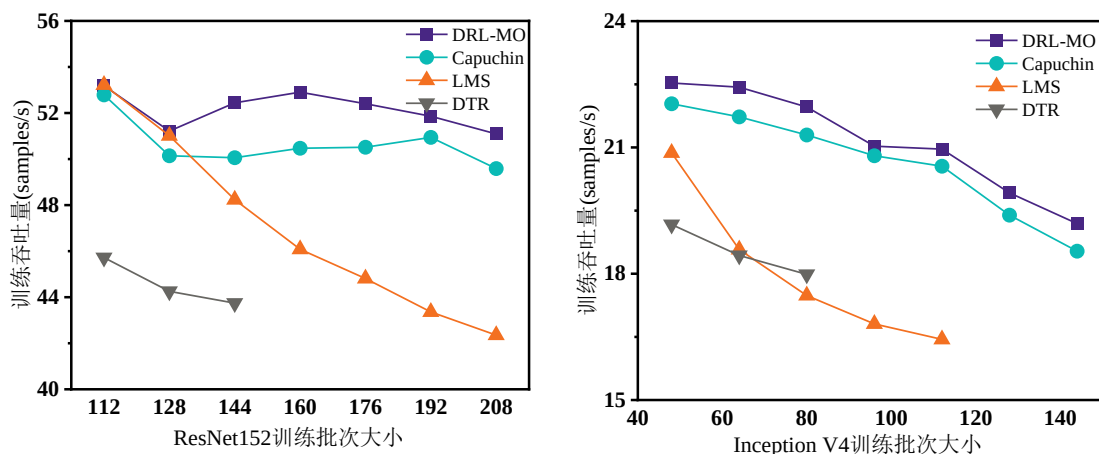


图 5.6 不同训练批次的 ResNet152 和 Inception V4 的训练吞吐量

由于 Inception V4^[21]和 ResNet152^[20]神经网络具有更大规模的数据流图，在本文的实验中，Checkmate^[10]和 MIP-MO 算法未能在 3 小时内获得卸载和重计算的策略。DRL-MO 可以利用通过深度强化学习获得的显存管理策略在短时间内获得高质量的策略。以 DTR^[13]，LMS^[9]和 Capuchin^[11]作为基准来证明 DRL-MO 的效果。从图 5.6 中可以看出，DRL-MO 显著减少了显存优化的额外开销。相比 Capuchin，LMS 和 DTR 算法，DRL-MO 的训练吞吐量分别达到 4.8%，27.5%和 22.1%的提升，平均情况下达到 18.1%的提升。

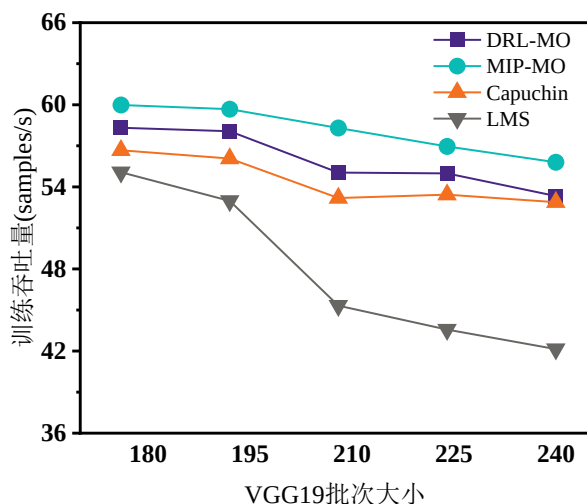


图 5.7 不同训练批次的 VGG19 的训练吞吐量

除了大规模 DNN, DRL-MO 算法也可以用于小规模深度神经网络。图 5.7 展示了 VGG19^[19]在不同的批次下, MIP-MO, DRL-MO, Capuchin^[11]和 LMS^[9]算法的训练吞吐量。从图中可知,相比于 Capuchin 和 LMS 算法, DRL-MO 的训练吞吐量分别达到了 3.5%, 26.1%的提升。但相比于全局搜索的 MIP-MO 算法, DRL-MO 的训练吞吐量平均低了 3.7%。

DRL-MO 比 MIP-MO 具有更低的算法运行时间开销。表 5.2 展示了这两种算法针对 VGG19 不同批次进行的优化, 它们都采用 CPU 进行运行。从表中可知, MIP-MO 算法的平均运行时间为 2336 秒。相比于数十个小时的 DNN 训练时间, 这种时间开销可以接受。而 DRL-MO 算法开销普遍低于 MIP-MO 算法, 平均情况下开销减少了 79 倍。

表 5.2 优化算法开销

VGG19 批次大小	MIP-MO 运行时间(s)	DRL-MO 运行时间(s)
208	2130	23.1
224	2949	27.7
240	4041	37.2

5.3.3 算子优化

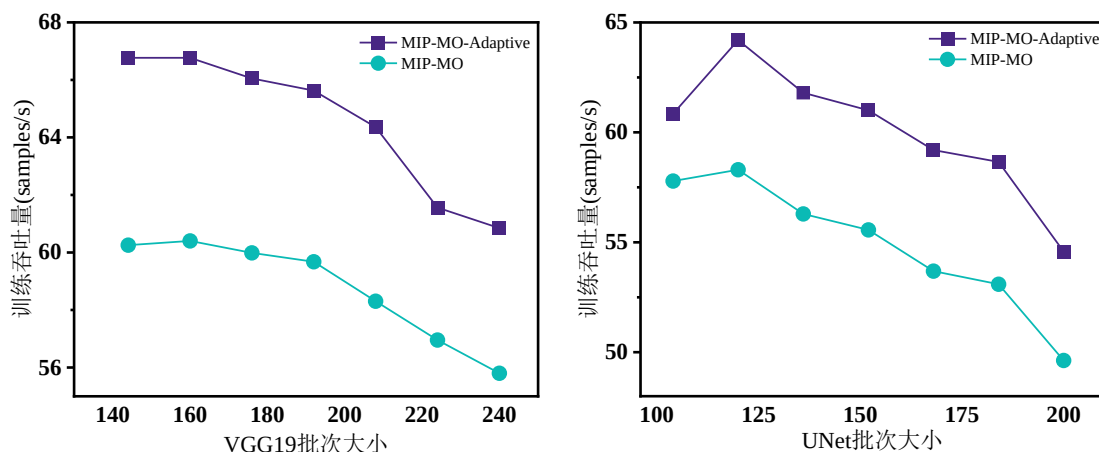


图 5.8 VGG19 和 UNet 的算子优化

由于不同的算子实现具有不同的显存需求和运行耗时, 本文在 MIP-MO 的实验中引入了算子优化模块, 验证算子选择的效果。MIP-MO 通过为计算节点选择显存需求较小的算子实现作为基准, 而 MIP-MO-Adaptive 则在 MIP-MO 的基础上引入了算子优化模块, 该模块根据空闲显存选择耗时最短的算子实现。从图 5.8 可知,

针对 VGG19^[19]和 UNet^[56]模型训练, 算子优化模块平均提升了 10%的训练性能, 这证明了算子优化在提高训练性能上的潜力。

表 5.3 VGG19 和 SqueezeNet 算子的差异性

VGG19 ^[19]			SqueezeNet ^[57]		
计算节点	计算耗时(s)	工作空间(MB)	计算节点	计算耗时(s)	工作空间(MB)
37	0.028	1768	62	0.029	1.95
	0.073	0		0.081	0
91	0.030	1898	90	0.009	1.38
	0.112	0		0.014	0
98	0.074	2220	115	0.017	0.39
	0.189	0		0.024	0

但不同的 DNN 对算子优化的敏感性不同, 该模块仅在部分 DNN 上存在性能提升的可能性。表 5.3 展示了 VGG19^[19]和 SqueezeNet^[57]数据流图中部分计算节点最快和最慢算子实现的计算耗时和额外的显存资源需求(工作空间)。从表中可知, VGG19 的计算节点需要通过增加大量显存占用才能使用最快的算子实现。而 SqueezeNet 的计算节点的最快算子实现仅需要少量的显存资源。Ant-NN 默认为计算节点配置显存需求小的算子实现, 因此, 算子优化模块并不能为 SqueezeNet 带来训练性能的进一步提升。

5.4 本章小结

本章首先描述了实验使用的训练平台 Ant-NN。然后, 本章介绍了实验的环境, 以及四种 DNN 训练的参数。接着, 本章介绍了使用的四种对比用例。最后, 本章展示并分析了 MIP-MO 和 DRL-MO 算法在四种 DNN 训练时的结果。

第6章 总结与展望

6.1 工作总结

随着 DNN 的发展, GPU 设备的扩容速度落后于训练的显存需求增加速度, 显存阻碍了 DNN 的进一步探索。卸载和重计算均能突破显存墙, 但同时给训练带来了延迟代价。对于这两种机制, 张量的显存优化代价不同, 结合机制为张量配置适当的卸载或重计算机制来减少延迟的影响。现有的结合工作未能充分考虑两种机制之间的相互影响, 可能导致为张量选择高代价的优化机制, 降低训练性能。针对此不足, 本文设计了两种显存优化算法, 主要工作如下:

为了获得高效的结合策略, 本文以训练延迟为基准, 对两种机制的显存优化代价建立了精确的混合显存优化统一代价模型。在此基础上, 本文提出了基于卸载和重计算混合机制的显存优化问题。该问题以引入的卸载, 预取和重计算的代价最小化为目标, 并通过整数规划的约束保证了张量计算, 卸载预取的正确性和显存容量限制。由于该问题是混合整数规划模型, 本文利用现有的整数规划求解器 Gurobi 设计了显存优化算法 MIP-MO, 并进一步设计了基于深度强化学习的显存优化算法 DRL-MO。MIP-MO 算法搜索时间长但能获得高质量的结合策略, 适用于小规模神经网络。DRL-MO 算法虽获得低质量的策略, 但搜索时间短, 适用于大规模神经网络。现有训练框架在实现卸载和重计算的机制方面存在一定的限制, 本文基于 CUDA 和 CUDNN 设计并实现了轻量的深度神经网络训练平台 Ant-NN。它由 5 个模块组成, 在有限的显存下为 DNN 训练透明地进行显存管理。本文在 Ant-NN 平台来验证 MIP-MO 和 DRL-MO 算法的效果。

实验结果表明, MIP-MO 相比于 LMS, Checkmate, Capuchin, 训练性能分别达到了 51%, 6.5%, 13.1% 的提升, 平均情况下达到了 23.5% 的提升。DRL-MO 相比于 LMS, DTR, Capuchin, 训练性能分别达到了 27.5%, 22.1%, 4.8% 的提升, 平均情况下达到了 18.1% 的提升。但相比于 MIP-MO 算法, DRL-MO 算法的搜索时间开销平均减少了 79 倍。经实验证明, 面向不同结构的 DNN, MIP-MO 和 DRL-MO 算法均能获得高效的结合策略, 减少了显存优化的代价。

6.2 工作展望

本文提出的显存优化算法减少了对 DNN 训练的延迟，但显存优化算法默认使用了显存需求小的算子实现。但算子存在显存需求大且耗时短的实现方式。在显存优化的实验中，DNN 在训练的部分时间内存在低显存占用的现象，且本文通过算子优化在部分 DNN 验证了算子选择的潜力。因此，以后的工作将进一步探讨算子选择与卸载重计算的协同优化。

参考文献

- [1] Zhu K, Guo H, Yan T, et al. Pass: Part-aware self-supervised pre-training for person re-identification[C]//European Conference on Computer Vision. Springer, 2022: 198-214.
- [2] Liang J, Fan C, Hou S, et al. Gaitedge: Beyond plain end-to-end gait recognition for better practicality[C]//European Conference on Computer Vision. Springer, 2022: 375-390.
- [3] Chen C, Zhao P, Li L, et al. Locally connected deep learning framework for industrial-scale recommender systems[C]//Proceedings of the 26th international conference on World Wide Web companion. ACM, 2017: 769-770.
- [4] Cheng H T, Koc L, Harmsen J, et al. Wide & deep learning for recommender systems[C]//Proceedings of the 1st workshop on deep learning for recommender systems. 2016: 7-10.
- [5] Baevski A, Zhou Y, Mohamed A, et al. wav2vec 2.0: A framework for self-supervised learning of speech representations[C]//Advances in neural information processing systems: Vol. 33. MIT Press, 2020: 12449-12460.
- [6] Graves A, Jaitly N. Towards end-to-end speech recognition with recurrent neural networks[C]//International conference on machine learning. PMLR, 2014: 1764-1772.
- [7] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners[C]//Advances in neural information processing systems. MIT Press, 2020: 1877-1901.
- [8] Sutskever I, Vinyals O, Le Q V. Sequence to sequence learning with neural networks[C]//Advances in neural information processing systems. MIT Press, 2014: 1-9.
- [9] Le T D, Imai H, Negishi Y, et al. Automatic gpu memory management for large neural models in tensorflow[C]//Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management. 2019: 1-13.
- [10] Jain P, Jain A, Nrusimha A, et al. Checkmate: Breaking the memory wall with optimal tensor rematerialization[C]//Proceedings of Machine Learning and Systems: Vol. 2. 2020: 497-511.
- [11] Peng X, Shi X, Dai H, et al. Capuchin: Tensor-based gpu memory management for deep learning[C]//Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, 2020: 891-905.
- [12] Raffel C, Shazeer N, Roberts A, et al. Exploring the limits of transfer learning with a unified text-to-text transformer[J]. The Journal of Machine Learning Research, 2020, 21(1): 5485-5551.
- [13] Kirisame M, Lyubomirsky S, Haan A, et al. Dynamic tensor rematerialization[C]//International Conference on Learning Representations. OpenReview, 2021: 1-31.
- [14] Lin S F, Chen Y J, Cheng H Y, et al. Tensor Movement Orchestration in Multi-GPU Training Systems[C]//2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE, 2023: 1140-1152.
- [15] Jung J, Kim J, Lee J. DeepUM: Tensor Migration and Prefetching in Unified Memory[C]//Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, 2023: 207-221.
- [16] LeCun Y, Bengio Y, Hinton G. Deep learning[J]. nature, 2015, 521(7553): 436-444.
- [17] Rumelhart D E, Hinton G E, Williams R J. Learning representations by back-propagating errors[J]. nature, 1986, 323(6088): 533-536.
- [18] Deng J, Dong W, Socher R, et al. Imagenet: A large-scale hierarchical image database[C]//2009 IEEE conference on computer vision and pattern recognition. IEEE, 2009: 248-255.

- [19] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[C]//International Conference on Learning Representations. OpenReview, 2015: 1-14.
- [20] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.
- [21] Szegedy C, Ioffe S, Vanhoucke V, et al. Inception-v4, inception-resnet and the impact of residual connections on learning[C]//Proceedings of the AAAI conference on artificial intelligence: Vol. 31. AAAI, 2017: 4278-4284.
- [22] Rhu M, Gimelshein N, Clemons J, et al. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design[C]//2016 49th Annual IEEE/ACM International Symposium on Microarchitecture. IEEE, 2016: 1-13.
- [23] Shriram S B, Garg A, Kulkarni P. Dynamic memory management for gpu-based training of deep neural networks[C]//2019 IEEE International Parallel and Distributed Processing Symposium. IEEE, 2019: 200-209.
- [24] Huang C C, Jin G, Li J. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping[C]//Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, 2020: 1341-1355.
- [25] Bae J, Lee J, Jin Y, et al. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks[C]//19th USENIX Conference on File and Storage Technologies. USENIX, 2021: 387-401.
- [26] Chen T, Xu B, Zhang C, et al. Training deep nets with sublinear memory cost[J]. arXiv preprint arXiv:1604.06174, 2016.
- [27] Wang L, Ye J, Zhao Y, et al. SuperNeurons: Dynamic GPU memory management for training deep neural networks[C]//Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming. ACM, 2018: 41-53.
- [28] Jin S, Zhang C, Jiang X, et al. COMET: A Novel Memory-Efficient Deep Learning Training Framework by Using Error-Bounded Lossy Compression[C]//Proceedings of International Conference on Very Large Databases. ACM, 2022: 886-899.
- [29] Chen C, Weng Q, Wang W, et al. Semi-dynamic load balancing: efficient distributed learning in non-dedicated environments[C]//Proceedings of the 11th ACM Symposium on Cloud Computing. ACM, 2020: 431-446.
- [30] Narayanan D, Phanishayee A, Shi K, et al. Memory-efficient pipeline-parallel dnn training[C]//International Conference on Machine Learning. PMLR, 2021: 7937-7947.
- [31] Chen C, Li M, Yang C. bbTopk: Bandwidth-Aware Sparse Allreduce with Blocked Sparsification for Efficient Distributed Training[C]//2023 IEEE 43rd International Conference on Distributed Computing Systems. IEEE, 2023: 1-11.
- [32] Jain A, Phanishayee A, Mars J, et al. Gist: Efficient data encoding for deep neural network training[C]//2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture. IEEE, 2018: 776-789.
- [33] Rajbhandari S, Rasley J, Ruwase O, et al. Zero: Memory optimizations toward training trillion parameter models[C]//International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2020: 1-16.
- [34] Chen P, He S, Zhang X, et al. Accelerating tensor swapping in gpus with self-tuning compression[J]. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(12): 4484-

- 4498.
- [35] Zhou Q, Wang H, Yu X, et al. MPRESS: Democratizing Billion-Scale Model Training on Multi-GPU Servers via Memory-Saving Inter-Operator Parallelism[C]//2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA). IEEE, 2023: 556-569.
 - [36] Hildebrand M, Khan J, Trika S, et al. Autotm: Automatic tensor movement in heterogeneous memory systems using integer linear programming[C]//Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. ACM, 2020: 875-890.
 - [37] Jin H, Wu W, Shi X, et al. TurboDL: Improving the CNN Training on GPU With Fine-Grained Multi-Streaming Scheduling[J]. IEEE Transactions on Computers, 2020, 70(4): 552-565.
 - [38] Gajda M, Trivella A, Mansini R, et al. An optimization approach for a complex real-life container loading problem[J]. Omega, 2022, 107: 102559.
 - [39] Morrison D R, Jacobson S H, Sauppe J J, et al. Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning[J]. Discrete Optimization, 2016, 19: 79-102.
 - [40] IBM CPLEX[EB/OL]. [2023-11-27]. <https://www.ibm.com/cn-zh/products/ilog-cplex-optimization-studio/cplex-optimizer>.
 - [41] Gurobi Optimization[EB/OL]. [2023-11-27]. <https://www.gurobi.com/>.
 - [42] Zhang C, Song W, Cao Z, et al. Learning to dispatch for job shop scheduling via deep reinforcement learning[C]//Advances in Neural Information Processing Systems. MIT Press, 2020: 1621-1632.
 - [43] Zhang D, Dai D, He Y, et al. RLScheduler: an automated HPC batch job scheduler using reinforcement learning[C]//International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2020: 1-15.
 - [44] Weisz G, Budzianowski P, Su P H, et al. Sample efficient deep reinforcement learning for dialogue systems with large action spaces[J]. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 2018, 26(11): 2083-2097.
 - [45] Derhami V, Paksima J, Khajah H. Web pages ranking algorithm based on reinforcement learning and user feedback[J]. Journal of AI and Data Mining, 2015, 3(2): 157-168.
 - [46] Lillicrap T P, Hunt J J, Pritzel A, et al. Continuous control with deep reinforcement learning[C]//International Conference on Learning Representations. ICLR, 2016: 1-14.
 - [47] Mnih V, Badia A P, Mirza M, et al. Asynchronous methods for deep reinforcement learning[C]//International conference on machine learning. PMLR, 2016: 1928-1937.
 - [48] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning[J]. nature, 2015, 518(7540): 529-533.
 - [49] Abadi M, Barham P, Chen J, et al. TensorFlow: a system for Large-Scale machine learning[C]//12th USENIX symposium on operating systems design and implementation. USENIX, 2016: 265-283.
 - [50] Paszke A, Gross S, Massa F, et al. Pytorch: An imperative style, high-performance deep learning library[C]//Advances in neural information processing systems. MIT Press, 2019: 1-12.
 - [51] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[C]//International Conference on Learning Representations. OpenReview, 2017: 1-10.
 - [52] Xu K, Hu W, Leskovec J, et al. How powerful are graph neural networks?[C]//International Conference on Learning Representations. OpenReview, 2018: 1-13.
 - [53] Chetlur S, Woolley C, Vandermersch P, et al. cuDNN: Efficient Primitives for Deep Learning[J].

- arXiv preprint arXiv:1410.0759, 2014.
- [54] Lu G, Zhang W, Wang Z. Optimizing depthwise separable convolution operations on gpus[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 33(1): 70-87.
- [55] Xu W, Sun Y, Fan S, et al. Accelerating Convolutional Neural Network by Exploiting Sparsity on GPUs[J]. ACM Transactions on Architecture and Code Optimization, 2023, 20(3): 1-26.
- [56] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation[J]. Medical Image Computing and Computer-Assisted Intervention, 2015, 9351: 234-241.
- [57] Iandola F N, Han S, Moskewicz M W, et al. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size[J]. arXiv preprint arXiv:1602.07360, 2016.